

Stage 07_00 — Experimental Design

1. Objetivo y alcance del Stage_07

1.1. Objetivo general

El objetivo del **Stage_07** es evaluar modelos de aprendizaje automático avanzados capaces de aprovechar la estructura temporal de los datos intradiarios del MNQ para predecir el **target operativo OPC**.

A diferencia de los modelos tabulares evaluados en el Stage_06, estos modelos recibirán como entrada una secuencia de observaciones históricas anteriores al instante de predicción. De esta manera, podrán analizar cómo evolucionan conjuntamente las features durante una ventana temporal determinada.

El problema se abordará como una tarea de **clasificación multiclase**, donde cada observación deberá ser asignada a una de las clases definidas por el target OPC.

La evaluación se realizará mediante un esquema de validación **walk-forward con ventana de entrenamiento expansiva**, respetando estrictamente el orden temporal de los datos.

1.2. Alcance

Durante este stage se desarrollará un protocolo experimental común para:

- construir ventanas temporales causales;
- mantener las mismas observaciones, features y targets entre modelos;
- entrenar modelos mediante los mismos folds walk-forward;
- evaluar diferentes arquitecturas de redes neuronales;
- comparar modelos utilizando las mismas métricas;
- analizar la estabilidad de los resultados entre períodos;
- guardar modelos, predicciones, probabilidades y métricas de manera estandarizada.

Los modelos iniciales considerados serán:

- Multilayer Perceptron — MLP;
- Convolutional Neural Network 1D — CNN 1D;
- Long Short-Term Memory — LSTM;
- Gated Recurrent Unit — GRU;
- Temporal Convolutional Network — TCN.

Cada modelo será desarrollado en una notebook independiente, pero todas las notebooks deberán seguir el mismo procedimiento experimental.

1.3. Principios metodológicos

El desarrollo del Stage_07 seguirá los siguientes principios:

1. Causalidad temporal

Cada predicción utilizará exclusivamente información disponible hasta el instante evaluado. Ninguna ventana podrá contener datos futuros.

2. Comparabilidad entre modelos

Todos los modelos deberán utilizar las mismas ventanas, features, targets, folds y métricas.

3. Validación fuera de muestra

El desempeño se evaluará sobre períodos posteriores a los utilizados para entrenar el modelo.

4. Prevención de leakage

El escalado, balanceo, selección de features y cualquier otra transformación deberán ajustarse exclusivamente con los datos de entrenamiento de cada fold.

5. Estabilidad temporal

No se seleccionará un modelo únicamente por obtener el mejor resultado en un período. También se analizará la consistencia de su desempeño entre los diferentes folds.

6. Complejidad progresiva

Los experimentos comenzarán con configuraciones controladas. La optimización extensa de hiperparámetros se realizará solamente sobre los modelos que demuestren señal predictiva estable.

1.4. Fuera del alcance

En este stage no se realizará todavía:

- el backtesting económico definitivo;
- la optimización de reglas de entrada y salida;
- la selección final de umbrales de probabilidad;
- la incorporación completa de costos, comisiones y slippage;
- la definición del tamaño de las posiciones;
- la evaluación de una estrategia lista para operar en producción.

Estas tareas deberán realizarse en un stage posterior, una vez identificados los modelos con mejor capacidad predictiva fuera de muestra.

1.5. Resultado esperado

Al finalizar el Stage_07 se espera disponer de:

- un conjunto de ventanas temporales validadas;
- modelos entrenados bajo un protocolo común;
- predicciones fuera de muestra para cada fold;
- probabilidades por clase;
- métricas generales y por clase;
- análisis de estabilidad temporal;
- una comparación consolidada entre arquitecturas;
- una selección reducida de modelos candidatos para la etapa de backtesting.

El resultado principal del stage no será determinar directamente si una estrategia es rentable, sino identificar qué modelos generan las señales predictivas más consistentes y confiables para el target OPC.

2. Punto de partida y resultados recibidos del Stage_06

2.1. Datasets predictivos disponibles

El Stage_06 permitió validar los datasets predictivos contruidos previamente para los targets BAR y OPC.

Para el desarrollo del Stage_07 se utilizarán únicamente los datasets correspondientes al target OPC, debido a que este target representa de forma directa el resultado operativo esperado de una posible entrada LONG o SHORT.

Las principales configuraciones disponibles son:

- OPC_full ;
- OPC_reduced_level ;
- OPC_reduced_no_level ;
- datasets con todos los regímenes intradiarios;
- datasets restringidos al régimen Regular.

El experimento principal del Stage_07 se iniciará utilizando:

```
TARGET_NAME = "opc_p50_h60_tp15_sl10"  
FEATURE_SET = "OPC_reduced_no_level"  
REGIME_SCOPE = "all_regimes"
```

Esta configuración fue seleccionada como punto de partida porque:

- utiliza un horizonte operativo intermedio de 60 minutos;
- evita comenzar con múltiples targets simultáneamente;
- reduce la cantidad de features;
- excluye variables asociadas directamente al nivel nominal del precio;
- conserva observaciones de todos los regímenes intradiarios.

Las otras configuraciones quedarán disponibles para experimentos posteriores de robustez.

2.2. Validación estructural y temporal

Durante el Stage_06 se verificó que los datasets seleccionados cumplieran con las condiciones necesarias para realizar experimentos predictivos.

Las principales validaciones realizadas fueron:

- índice temporal correctamente ordenado;
- ausencia de índices duplicados;
- correspondencia entre features y target;
- ausencia de valores infinitos;
- control de valores faltantes;
- identificación de columnas constantes;
- separación entre features predictivas y variables auxiliares;
- revisión de posibles variables con información futura;
- consistencia temporal entre los diferentes datasets.

También se verificó que las features reducidas fueran subconjuntos coherentes de las versiones completas.

Estas validaciones permiten utilizar los datasets del Stage_06 como base del Stage_07 sin repetir todo el proceso de auditoría desde cero.

No obstante, antes de construir las secuencias temporales se volverán a ejecutar controles específicos sobre:

- continuidad temporal;
- cambios de día;
- cambios de contrato;
- longitud completa de las ventanas;
- correspondencia entre el último instante de la ventana y el target.

2.3. Auditoría de leakage

El Stage_06 incluyó una auditoría destinada a evitar que los modelos recibieran información que no estaría disponible en el momento real de la predicción.

Se excluyeron o identificaron como no predictivas variables relacionadas con:

- máximos o mínimos futuros;
- excursiones futuras;
- MFE y MAE;
- eventos futuros de TP o SL;
- otros targets;
- información del resultado final del trade;
- variables utilizadas únicamente para construir el dataset;
- identificación explícita del split temporal.

El Stage_07 mantendrá las mismas restricciones.

Además, cualquier transformación adicional, como escalado, imputación o balanceo de clases, deberá ajustarse exclusivamente con los datos de entrenamiento correspondientes a cada fold.

2.4. Esquema walk-forward validado

El Stage_06 definió y validó tres folds walk-forward con ventana de entrenamiento expansiva:

Fold	Período de entrenamiento	Período de validación
WF_01	2020–2021	2022
WF_02	2020–2022	2023
WF_03	2020–2023	2024

Este esquema mantiene el orden cronológico y simula el proceso de entrenar un modelo con la información histórica disponible para evaluarlo sobre un período posterior.

Las principales características verificadas fueron:

- ausencia de solapamiento entre train y validation;
- entrenamiento siempre anterior a la validación;
- crecimiento progresivo del conjunto de entrenamiento;
- presencia de las clases del target en los folds;
- cobertura temporal esperada;
- consistencia de los índices.

El Stage_07 conservará exactamente estos folds para que los resultados de los nuevos modelos sean comparables con los experimentos anteriores.

2.5. Resultados del análisis de Mutual Information

El análisis univariado realizado en el Stage_06 mostró que el target OPC contiene relaciones predictivas detectables con distintas variables.

En la versión completa del dataset, las mayores puntuaciones de Mutual Information estuvieron asociadas principalmente con:

- máximos y mínimos móviles;
- posición y amplitud del rango;
- volatilidad reciente;
- estadísticas de rango de las barras;
- variables calculadas sobre ventanas de 30, 60 y 90 minutos.

En particular, algunas features relacionadas con el nivel del precio presentaron valores de Mutual Information considerablemente superiores al resto.

Sin embargo, esta señal debe interpretarse con cautela, porque podría reflejar:

- crecimiento nominal del índice a través de los años;
- diferencias entre contratos;
- cambios estructurales del mercado;
- asociaciones temporales que no necesariamente generalicen.

Por esta razón, el Stage_07 comenzará con la versión:

OPC_reduced_no_level

Las features de nivel podrán incorporarse posteriormente como experimento comparativo.

2.6. Limitaciones de los experimentos anteriores

Los experimentos del Stage_06 utilizaron principalmente una representación tabular de los datos.

En esta representación, cada fila contiene el estado de las features en un instante determinado:

$$X_t \rightarrow y_t$$

Aunque muchas features ya resumen información histórica mediante ventanas móviles, el modelo no recibe explícitamente la secuencia completa que produjo esos valores.

El Stage_07 modificará esta representación para entregar al modelo una sucesión de observaciones:

$$[X_{(t-L+1)}, \dots, X_{(t-1)}, X_t] \rightarrow y_t$$

donde L representa la longitud de la ventana temporal.

Esto permitirá evaluar si los modelos pueden aprender patrones relacionados con:

- evolución de la volatilidad;
- aceleración o desaceleración del precio;
- cambios en momentum;
- formación de rangos;
- transiciones entre estados del mercado;
- interacción temporal entre varias features.

2.7. Decisión de continuidad

A partir de los resultados del Stage_06, el Stage_07 continuará con las siguientes decisiones iniciales:

TARGET_NAME = "opc_p50_h60_tp15_sl10"

FEATURE_SET = "OPC_reduced_no_level"

REGIME_SCOPE = "all_regimes"

WALK_FORWARD_FOLDS = [
 "WF_01",
 "WF_02",
 "WF_03",
]

PRIMARY_LOOKBACK = 60

Estas decisiones representan el experimento principal y no una selección definitiva.

Luego de obtener una primera comparación entre modelos se podrán evaluar:

- ventanas de 30 y 90 minutos;
- features con nivel;
- datasets completos;
- régimen Regular;

- otros horizontes OPC;
- ajustes limitados de hiperparámetros.

El objetivo inicial será mantener controlado el número de experimentos y determinar si la incorporación explícita de secuencias temporales mejora la capacidad predictiva fuera de muestra.

3. Definición del problema predictivo y del target OPC

3.1. Tipo de problema

El Stage_07 se plantea como un problema de **clasificación multiclase supervisada**.

Para cada instante intradiario (t), el modelo recibirá una ventana con información histórica del mercado y deberá estimar cuál de los posibles resultados operativos definidos por el target OPC ocurrirá posteriormente.

La relación general será:

Secuencia histórica disponible hasta $t \rightarrow$ clase OPC correspondiente a t
Formalmente:

$[X_{(t-L+1)}, \dots, X_{(t-1)}, X_t] \rightarrow y_t$
donde:

- L es la longitud de la ventana histórica;
- X_t contiene las features disponibles en el instante t ;
- y_t es la clase OPC calculada utilizando el comportamiento futuro del precio;
- el modelo solamente recibe información disponible hasta t .

El comportamiento futuro se utiliza únicamente para construir el target durante el desarrollo histórico. Nunca debe formar parte de las features utilizadas por el modelo.

3.2. Target principal

El target seleccionado inicialmente es:

TARGET_NAME = "opc_p50_h60_tp15_sl10"

El nombre representa la siguiente configuración:

Componente	Significado
opc	Target operativo combinado
p50	Threshold direccional correspondiente al percentil 50
h60	Horizonte futuro máximo de 60 minutos

Componente	Significado
tp15	Take Profit equivalente a 1.5 veces el threshold
sl10	Stop Loss equivalente a 1.0 vez el threshold

Es importante aclarar que **tp15** y **sl10** no representan 15 y 10 puntos fijos.

Representan multiplicadores aplicados al threshold correspondiente al régimen intradiario de cada observación:

$$TP = \text{threshold} \times 1.5$$

$$SL = \text{threshold} \times 1.0$$

El target direccional utilizado como base es:

$$\text{dir_p50_h60}$$

Este target determina si el movimiento futuro es suficientemente significativo y en qué dirección, utilizando un threshold específico para cada régimen intradiario.

El target DIR no utiliza niveles de Take Profit ni Stop Loss para realizar su clasificación.

Los siguientes valores se incluyen únicamente como referencia conceptual. Los thresholds exactos utilizados por cada observación deberán obtenerse de los metadatos y datasets generados en los Stage_03 y Stage_04.

Régimen	Threshold aproximado
Overnight	22.25–24.75 puntos
Pre-market	57.00–63.25 puntos
Opening	63.75 puntos
Regular	41.00 puntos

Por ejemplo, para una observación perteneciente al régimen Regular:

$$\text{Threshold DIR} = 41.0 \text{ puntos}$$

$$\text{Distancia del TP} = 41.0 \times 1.5 = 61.5 \text{ puntos}$$

$$\text{Distancia del SL} = 41.0 \times 1.0 = 41.0 \text{ puntos}$$

Por lo tanto, los niveles operativos no son constantes. Cambian según el régimen intradiario y el threshold correspondiente.

Las barras futuras empleadas para construir el target son:

$$t+1, t+2, \dots, t+60$$

El precio y las features del instante **t** pueden formar parte de la entrada, pero ninguna barra posterior a **t** podrá ser utilizada por el modelo.

3.3. Relación entre los targets DIR, BAR y OPC

El target OPC combina la dirección significativa definida por DIR con el resultado operativo evaluado mediante barreras.

La secuencia conceptual es:

DIR → determina la dirección significativa
 BAR → evalúa el resultado de las barreras
 OPC → combina dirección y resultado operativo

Target DIR

El target `dir_p50_h60` determina si existe un movimiento suficientemente significativo dentro del horizonte futuro y en qué dirección ocurre.

Conceptualmente, sus posibles resultados son:

LONG
 SHORT
 Sin dirección significativa

La clasificación se realiza comparando el movimiento futuro con el threshold correspondiente al régimen intradiario de la observación.

Target BAR

El target `bar_p50_h60_tp15_sl10` evalúa el resultado de una operación utilizando barreras calculadas a partir del threshold:

Distancia del TP = $\text{threshold} \times 1.5$
 Distancia del SL = $\text{threshold} \times 1.0$

Las posiciones de las barreras dependen de la dirección de la operación.

Para una operación LONG:

TP = precio de entrada + $\text{threshold} \times 1.5$
 SL = precio de entrada - $\text{threshold} \times 1.0$

Para una operación SHORT:

TP = precio de entrada - $\text{threshold} \times 1.5$
 SL = precio de entrada + $\text{threshold} \times 1.0$

Luego se evalúa cuál de las barreras se alcanza primero dentro del horizonte futuro.

Target OPC

El target `opc_p50_h60_tp15_sl10` combina la dirección determinada por DIR con el resultado operativo evaluado mediante las barreras.

Las clases conceptuales son:

Clase	Interpretación
LONG_TP	La dirección es LONG y el precio alcanza primero el TP
LONG_SL	La dirección es LONG y el precio alcanza primero el SL
SHORT_TP	La dirección es SHORT y el precio alcanza primero el TP

Clase	Interpretación
SHORT_SL	La dirección es SHORT y el precio alcanza primero el SL
NO_TRADE	No existe una condición operativa válida

Una explicación sencilla sería:

```

LONG_TP    → una operación LONG habría resultado ganadora
LONG_SL    → una operación LONG habría resultado perdedora
SHORT_TP   → una operación SHORT habría resultado ganadora
SHORT_SL   → una operación SHORT habría resultado perdedora
NO_TRADE   → no se habría abierto una operación válida

```

La lógica exacta utilizada para asignar `NO_TRADE` deberá verificarse en el código y los metadatos del Stage_04.

En particular, se comprobará si `NO_TRADE` representa únicamente la ausencia de una dirección significativa o si también incluye casos en los que ninguna barrera fue alcanzada dentro del horizonte.

El modelo no predice directamente un precio futuro ni utiliza niveles fijos de puntos. Predice cuál de estos cinco escenarios operativos es más probable.

3.4. Codificación numérica

Los modelos neuronales necesitan representar las clases mediante valores numéricos enteros.

La codificación exacta deberá obtenerse de los metadatos generados en el Stage_04 o verificarse directamente sobre el dataset antes del entrenamiento.

No se deberá asumir manualmente que una clase corresponde a un número determinado.

La configuración podrá almacenarse mediante un diccionario como el siguiente:

```

OPC_CLASS_MAPPING = {
    "LONG_TP": 0,
    "LONG_SL": 1,
    "SHORT_TP": 2,
    "SHORT_SL": 3,
    "NO_TRADE": 4,
}

```

Este diccionario representa únicamente la estructura esperada.

Antes de utilizarlo deberá comprobarse que coincide con la codificación real del dataset.

También se generará el diccionario inverso:

```

OPC_CLASS_NAMES = {
    class_id: class_name
    for class_name, class_id in OPC_CLASS_MAPPING.items()
}

```

Esto permitirá convertir las predicciones numéricas nuevamente en nombres interpretables.

3.5. Salida esperada del modelo

El modelo deberá producir una probabilidad para cada una de las cinco clases.

Ejemplo:

```
LONG_TP:    0.48
LONG_SL:    0.12
SHORT_TP:   0.14
SHORT_SL:   0.08
NO_TRADE:   0.18
```

La salida representa:

```
P(y = clase | ventana histórica)
```

Para cada muestra:

```
y_probability.shape = (5,)
```

Para todo el conjunto de datos:

```
y_probability.shape = (
    n_samples,
    n_classes,
)
```

La predicción directa del modelo será normalmente la clase con mayor probabilidad:

```
predicted_class = y_probability.argmax()
```

En el ejemplo anterior, la predicción sería:

```
LONG_TP
```

Sin embargo, esto no significa automáticamente que se ejecutará una operación.

La decisión operativa podrá requerir posteriormente un nivel mínimo de confianza.

Por ejemplo:

```
Operar solamente si:
```

```
P(LONG_TP) >= 0.60
```

La selección de estos umbrales de probabilidad no pertenece todavía a esta etapa.

3.6. Diferencia entre predicción y decisión de trading

Es importante separar dos procesos:

```
Modelo predictivo:
estima las probabilidades de las clases OPC.
```

```
Regla operativa:
```

decide si se abre o no una operación utilizando esas probabilidades.

Por ejemplo, el modelo podría generar:

```
LONG_TP = 0.35  
NO_TRADE = 0.30  
SHORT_TP = 0.20  
LONG_SL = 0.10  
SHORT_SL = 0.05
```

La clase más probable sería `LONG_TP`, pero una probabilidad de `0.35` podría ser insuficiente para justificar una operación.

Por esta razón, en el `Stage_07` se evaluará principalmente la calidad predictiva y probabilística del modelo.

La traducción definitiva de las probabilidades a operaciones se realizará posteriormente durante la etapa de backtesting.

3.7. Unidad de observación

Cada muestra del dataset estará asociada con un único timestamp t .

Para ese timestamp se tendrá:

Entrada:
ventana histórica que termina en t .

Salida:
clase OPC correspondiente a t .

Por ejemplo, para una ventana de 60 minutos:

Timestamp objetivo: 10:30

Entrada:
features entre 09:31 y 10:30.

Target:
resultado OPC calculado desde 10:31 hasta un máximo de 11:30.

La ventana de entrada termina exactamente en el instante de decisión.

El horizonte utilizado para construir el target comienza en la barra siguiente.

3.8. Solapamiento temporal y separación de los conjuntos

Debido a que el target utiliza un horizonte futuro de 60 minutos, las etiquetas de observaciones consecutivas comparten gran parte del mismo período futuro.

Por ejemplo:

Target de 10:30:
utiliza el comportamiento futuro entre 10:31 y 11:30.

Target de 10:31:

utiliza el comportamiento futuro entre 10:32 y 11:31.

Por lo tanto, ambas etiquetas están relacionadas porque fueron construidas utilizando casi el mismo tramo futuro.

Este solapamiento no representa un problema cuando las observaciones pertenecen al mismo conjunto, por ejemplo, cuando ambas forman parte del entrenamiento.

El problema aparecería si una observación quedara en entrenamiento y otra observación cercana, que comparte prácticamente el mismo futuro, quedara en validación.

Para evitar esta contaminación, las divisiones temporales no se realizarán dentro de una misma jornada. Los conjuntos se separarán utilizando **días de trading completos y consecutivos**.

Como los targets fueron construidos sin cruzar días, se garantiza que:

Los targets del último día de entrenamiento
solo utilizan información futura de ese mismo día.

Los targets del primer día de validación
solo utilizan información futura del nuevo día.

De esta manera, entrenamiento y validación no comparten el mismo período futuro y no es necesario eliminar 60 minutos de información en cada jornada.

División interna para early stopping

Dentro del período de entrenamiento de cada fold walk-forward se realizará una división adicional:

Train interno:
se utiliza para ajustar los pesos del modelo.

Validation interna:
se utiliza para controlar el entrenamiento y aplicar early stopping.

Por ejemplo, para el fold **WF_01** :

Train general del fold:
2020-2021

Train interno:
2020 hasta septiembre de 2021

Validation interna:
octubre hasta diciembre de 2021

Validación walk-forward:
2022

Las fechas exactas de la validación interna se definirán posteriormente, pero siempre se respetarán las siguientes reglas:

- la división será cronológica;
- se utilizarán días completos;
- no se mezclarán observaciones de una misma jornada;
- la validación interna siempre será posterior al train interno;
- la validación walk-forward permanecerá completamente fuera del entrenamiento.

Entrenamiento definitivo de cada fold

La validación interna se utilizará para determinar aspectos como:

- la mejor cantidad de épocas;
- el momento adecuado para detener el entrenamiento;
- la presencia de sobreajuste;
- la configuración inicial del modelo.

Una vez determinada la mejor cantidad de épocas, el modelo del fold se volverá a entrenar desde cero utilizando todo el período de entrenamiento disponible.

Por ejemplo:

Primera ejecución:

Train interno:

2020 hasta septiembre de 2021

Validation interna:

octubre hasta diciembre de 2021

Mejor época identificada:

14

Luego se realizará:

Entrenamiento definitivo del fold:

Datos:

todo 2020–2021

Cantidad de épocas:

14

Evaluación fuera de muestra:

todo 2022

Así, los datos utilizados inicialmente como validación interna también se aprovechan en el entrenamiento definitivo del fold.

Aplicación dentro del walk-forward

La estructura general será:

WF_01:

Train general 2020–2021

Validación walk-forward 2022

WF_02:
Train general 2020–2022
Validación walk-forward 2023

WF_03:
Train general 2020–2023
Validación walk-forward 2024

Dentro del train general de cada fold se realizará temporalmente la separación entre train interno y validation interna para controlar el entrenamiento.

Posteriormente, el modelo se volverá a entrenar con todo el train general y se evaluará una única vez sobre el año de validación walk-forward.

Test final

Los períodos posteriores utilizados como test final permanecerán completamente reservados durante la comparación de modelos.

Conceptualmente:

Train interno + validation interna:
desarrollo del modelo.

Train general completo:
entrenamiento definitivo de cada fold.

Año siguiente:
validación walk-forward.

Período final reservado:
test definitivo del modelo seleccionado.

El test final no se utilizará para:

- elegir la arquitectura;
- ajustar hiperparámetros;
- seleccionar la cantidad de épocas;
- comparar modelos;
- definir umbrales operativos.

Su función será realizar una última evaluación completamente fuera de muestra después de seleccionar el modelo final.

En resumen, el solapamiento entre etiquetas consecutivas se permitirá dentro de un mismo conjunto, pero se evitará que atravesase las fronteras entre entrenamiento, validación interna, validación walk-forward y test.

3.9. Desbalance entre clases

Las cinco clases OPC probablemente no tendrán la misma cantidad de observaciones.

Es posible que:

- `NO_TRADE` sea la clase más frecuente;
- algunas clases de TP tengan mayor frecuencia que sus equivalentes de SL;
- la distribución cambie entre años;
- la distribución cambie entre regímenes;
- algunos folds tengan mayor dificultad que otros.

Por esta razón, antes de entrenar se deberá verificar:

```
Cantidad de muestras por clase
Porcentaje de cada clase
Distribución por fold
Distribución por año
Distribución por régimen
```

El modelo no deberá evaluarse únicamente mediante accuracy, porque podría obtener un resultado aparentemente bueno prediciendo repetidamente la clase mayoritaria.

3.10. Validaciones obligatorias

Antes de comenzar cualquier entrenamiento se deberá comprobar que:

1. El target existe en el dataset.
2. El target no contiene valores fuera de la codificación OPC.
3. El target no contiene valores faltantes en las muestras utilizadas.
4. Todas las clases esperadas están presentes.
5. La codificación numérica coincide con los metadatos.
6. Cada timestamp tiene una única clase.
7. El target está correctamente alineado con el último instante de cada ventana.
8. La ventana de entrada no contiene barras posteriores al timestamp objetivo.
9. El horizonte futuro utilizado para construir el target no cruza días de trading.
10. Las variables utilizadas para construir el target no están incluidas entre las features.
11. El threshold correspondiente a cada observación coincide con su régimen intradiario.
12. Los multiplicadores de TP y SL se aplicaron correctamente.
13. La lógica de asignación de `NO_TRADE` coincide con la definición utilizada en el Stage_04.

3.11. Configuración inicial

La configuración correspondiente a este punto será:

```
TARGET_CONFIG = {
    "target_name": "opc_p50_h60_tp15_sl10",
    "base_directional_target": "dir_p50_h60",
    "barrier_target": "bar_p50_h60_tp15_sl10",
    "problem_type": "multiclass_classification",
    "target_horizon_minutes": 60,
    "threshold_percentile": 50,
```

```

        "threshold_scope": "regime",
        "take_profit_multiplier": 1.5,
        "stop_loss_multiplier": 1.0,
        "n_classes": 5,
        "class_names": [
            "LONG_TP",
            "LONG_SL",
            "SHORT_TP",
            "SHORT_SL",
            "NO_TRADE",
        ],
    }

```

No se almacenará un único valor numérico de threshold, porque este depende del régimen intradiario correspondiente a cada observación.

La codificación numérica definitiva de las clases se agregará después de comprobar los metadatos reales del target.

3.12. Decisión metodológica

El problema principal del Stage_07 queda definido como:

Predecir las probabilidades de los cinco resultados operativos OPC utilizando una secuencia causal de datos intradiarios disponible hasta el instante de decisión.

El objetivo no será predecir directamente el precio futuro ni decidir todavía una operación definitiva.

El modelo deberá aprender a diferenciar entre:

- Operaciones LONG favorables
- Operaciones LONG desfavorables
- Operaciones SHORT favorables
- Operaciones SHORT desfavorables
- Situaciones sin operación válida

Estas probabilidades constituirán la señal predictiva que posteriormente será transformada en reglas operativas y evaluada mediante backtesting.

4. Selección del dataset experimental y definición de features

4.1. Objetivo

El objetivo de este punto es definir el dataset que se utilizará como base para construir las ventanas temporales del Stage_07.

También se establecerá qué columnas podrán ingresar como variables predictivas y qué columnas deberán conservarse únicamente como información auxiliar o de contexto.

Esta separación es necesaria para evitar que los modelos reciban:

- información futura;
- columnas utilizadas para construir el target;
- otros targets;
- identificadores sin valor predictivo;
- información perteneciente al split temporal;
- variables que puedan introducir leakage.

Todos los modelos del Stage_07 deberán utilizar la misma selección de observaciones y features para garantizar una comparación justa entre arquitecturas.

4.2. Dataset experimental principal

El experimento principal utilizará el dataset:

```
DATASET_NAME = (  
    "opc_p50_h60_tp15_sl10"  
    "__OPC_reduced_no_level"  
    "__all_regimes"  
)
```

Esta configuración contiene:

Target:
opc_p50_h60_tp15_sl10

Familia:
OPC

Conjunto de features:
reduced_no_level

Cobertura intradiaria:
all_regimes

El dataset contiene una fila por timestamp y reúne:

- el target OPC;
- las features predictivas seleccionadas;
- variables de contexto temporal;
- columnas auxiliares necesarias para validar y reconstruir los experimentos.

No todas las columnas del dataset ingresarán directamente al modelo.

Antes de construir las ventanas se separarán explícitamente:

```
Features predictivas  
Target  
Variables de contexto  
Columnas auxiliares  
Columnas excluidas
```

4.3. Justificación de la selección

Se utilizará inicialmente la versión `OPC_reduced_no_level` para comenzar con una representación más controlada del problema.

La versión reducida permite:

- trabajar con una menor cantidad de features;
- disminuir la dimensionalidad de las ventanas;
- reducir el consumo de memoria;
- reducir el costo de entrenamiento;
- limitar el riesgo de sobreajuste;
- facilitar la comparación entre modelos.

La variante `no_level` excluye las features asociadas directamente con el nivel nominal del precio, como máximos y mínimos móviles expresados en puntos absolutos.

Estas variables mostraron valores elevados de Mutual Information en el Stage_06. Sin embargo, también podrían capturar:

- el crecimiento nominal del MNQ entre años;
- diferencias entre contratos;
- cambios estructurales del mercado;
- asociaciones temporales que no necesariamente generalicen.

Por esta razón, el experimento principal comenzará sin features de nivel.

La versión `OPC_reduced_level` se conservará para un experimento posterior de robustez, con el objetivo de comprobar si las variables de nivel mejoran realmente el desempeño fuera de muestra.

También se utilizarán inicialmente todos los regímenes intradiarios:

```
REGIME_SCOPE = "all_regimes"
```

Esto permitirá:

- aprovechar una mayor cantidad de observaciones;
- evaluar el comportamiento general del modelo;
- permitir que el modelo aprenda diferencias entre horarios;
- analizar posteriormente los resultados por régimen.

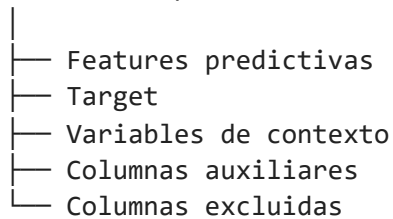
Los experimentos restringidos al régimen Regular se realizarán únicamente después de obtener una línea base con todos los regímenes.

4.4. Tipos de columnas del dataset

Antes de construir las ventanas temporales, todas las columnas del dataset deberán clasificarse según su función dentro del experimento.

La separación general será:

Dataset completo



Esta clasificación permitirá controlar exactamente qué información recibe el modelo y qué información se conserva únicamente para organizar, validar o analizar los resultados.

4.4.1. Features predictivas

Son las variables que el modelo utilizará como entrada para realizar la predicción.

Formalmente:

X_t = features disponibles en el instante t

Estas columnas podrán incluir información causal relacionada con:

- retornos históricos;
- rangos de precio;
- volatilidad;
- momentum;
- posición dentro de rangos móviles;
- estadísticas históricas de las barras;
- volumen relativo;
- indicadores técnicos;
- contexto temporal permitido.

Las features deberán cumplir las siguientes condiciones:

1. Estar disponibles en el instante de predicción.
2. No utilizar información posterior al timestamp t .
3. No contener valores derivados del resultado futuro.
4. No incluir otros targets.
5. No identificar directamente el split temporal.
6. Haber sido aprobadas durante la auditoría de leakage.

Las ventanas temporales se construirán exclusivamente con estas columnas:

(features de $t-L+1$, ..., features de t) $\rightarrow$ target de t

4.4.2. Target

El target es la variable que el modelo deberá predecir.

Para el experimento principal:

TARGET_COLUMN = "opc_p50_h60_tp15_sl10"

Esta columna:

- no forma parte de las features;
- no se incluye dentro de la ventana de entrada;
- se utiliza únicamente como salida esperada;
- debe estar alineada con el último timestamp de cada ventana.

La relación será:

Ventana de features que termina en $t \rightarrow$ target OPC de t

4.4.3. Variables de contexto

Las variables de contexto describen las condiciones en las que se encuentra cada observación.

Ejemplos:

```
minute_of_day  
regime_id  
contract  
year  
quarter
```

Estas variables pueden cumplir dos funciones diferentes:

Contexto predictivo

Son variables que pueden ingresar al modelo porque están disponibles en tiempo real y pueden aportar señal.

Por ejemplo:

```
minute_of_day  
regime_id
```

Estas variables permiten que el modelo aprenda que el comportamiento del mercado puede cambiar según el horario o el régimen intradiario.

Contexto analítico

Son variables que se conservan para analizar los resultados, pero no necesariamente ingresan al modelo.

Por ejemplo:

```
year  
contract  
dataset_split
```

Estas columnas pueden utilizarse posteriormente para estudiar:

- rendimiento por año;
- rendimiento por contrato;
- estabilidad entre períodos;

- diferencias entre regímenes;
- distribución de clases.

La inclusión de cada variable de contexto como feature deberá definirse explícitamente. No se asumirá que todas las variables de contexto ingresan automáticamente al modelo.

4.4.4. Columnas auxiliares

Son columnas necesarias para construir, validar o reconstruir los experimentos, pero que no deben ingresar al modelo.

Ejemplos posibles:

```
datetime
date
trading_date
dataset_split
fold_id
target_name
contract_target
```

Estas columnas podrán utilizarse para:

- mantener el orden temporal;
- identificar días de trading;
- evitar que las ventanas crucen jornadas;
- detectar cambios de contrato;
- construir los folds walk-forward;
- guardar predicciones con su timestamp;
- analizar resultados por período;
- reconstruir una operación durante el backtesting.

Las columnas auxiliares deben conservarse asociadas a cada muestra, pero separadas de la matriz predictiva `X`.

4.4.5. Columnas excluidas

Son variables que no deberán utilizarse como entrada del modelo.

Se excluirán:

- el target principal;
- otros targets DIR, BAR u OPC;
- variables futuras;
- máximos y mínimos futuros;
- MFE y MAE;
- indicadores de TP o SL alcanzados;
- duración futura de la operación;
- resultado final del trade;
- columnas utilizadas para construir el target;

- identificadores del fold;
- etiquetas de train, validation o test;
- variables detectadas como leakage;
- columnas constantes;
- columnas duplicadas;
- identificadores sin significado predictivo.

Ejemplos conceptuales:

```
future_high_max
future_low_min
mfe_pts
mae_pts
tp_hit
sl_hit
target_result
dataset_split
fold_id
```

La lista exacta dependerá de las columnas reales presentes en el dataset.

4.4.6. Separación final

Antes de construir las ventanas se crearán listas explícitas:

```
FEATURE_COLUMNS = [...]

TARGET_COLUMN = "opc_p50_h60_tp15_sl10"

CONTEXT_COLUMNS = [...]

AUXILIARY_COLUMNS = [...]

EXCLUDED_COLUMNS = [...]
```

La matriz de entrada deberá construirse únicamente con:

```
X = df[FEATURE_COLUMNS]
```

El target deberá extraerse mediante:

```
y = df[TARGET_COLUMN]
```

La información auxiliar deberá mantenerse separada:

```
metadata = df[AUXILIARY_COLUMNS]
```

4.4.7. Regla principal

La regla general será:

El modelo solamente recibe las features predictivas.

El target se utiliza como respuesta correcta.

Las variables auxiliares organizan el experimento.

Las variables excluidas nunca ingresan al modelo.

La clasificación exacta de las columnas se verificará directamente sobre el dataset antes de generar las ventanas temporales.

4.5. Features predictivas

Las features predictivas son las variables que ingresarán al modelo para estimar la clase OPC correspondiente a cada timestamp.

Para el experimento principal se utilizará el conjunto de features definido en el Stage_06 como:

`FEATURE_SET = "OPC_reduced_no_level"`

La lista definitiva no se reconstruirá manualmente dentro del Stage_07. Se obtendrá directamente de los metadatos guardados durante los Stage_05 y Stage_06.

Esto permitirá garantizar que:

- se utilicen las mismas features ya validadas;
- no se incorporen columnas accidentalmente;
- se mantenga la trazabilidad entre stages;
- la selección sea idéntica para todos los modelos.

4.5.1. Significado de `reduced_no_level`

La configuración `reduced_no_level` contiene un conjunto reducido de features causales y excluye principalmente variables que representan directamente el nivel nominal del precio.

Ejemplos de variables de nivel que no se utilizarán inicialmente:

```
rolling_high_30m
rolling_high_60m
rolling_high_90m
rolling_low_30m
rolling_low_60m
rolling_low_90m
```

Estas variables indican en qué nivel absoluto se encontraba el MNQ.

Por ejemplo:

`rolling_high_60m = 21 450 puntos`

El problema de estas variables es que pueden permitir que el modelo identifique indirectamente:

- el año;
- el contrato;
- períodos específicos del mercado;

- el crecimiento nominal del índice.

Esto podría producir buenos resultados históricos sin garantizar una correcta generalización futura.

La expresión `no_level` no significa que todas las features expresadas en puntos sean eliminadas. Significa principalmente que se excluyen variables que representan directamente el nivel absoluto del precio.

4.5.2. Familias de features

Las features predictivas estarán organizadas en las siguientes familias.

Retornos

Representan cambios relativos del precio durante diferentes períodos.

Ejemplos conceptuales:

```
ret_1m  
ret_5m  
ret_10m  
ret_15m  
ret_30m  
ret_60m
```

Estas variables permiten estudiar:

- dirección reciente;
- intensidad del movimiento;
- continuidad;
- reversión;
- aceleración del precio.

Los retornos relativos son especialmente útiles porque son más comparables entre distintos niveles nominales del MNQ.

Momentum

Representan la persistencia o fuerza del movimiento reciente.

Ejemplos conceptuales:

```
momentum_5m  
momentum_15m  
momentum_30m  
roc_10m  
roc_30m
```

Estas features pueden ayudar al modelo a distinguir entre:

```
Movimiento que está comenzando  
Movimiento que se está fortaleciendo
```

Movimiento que está perdiendo fuerza
Movimiento que podría revertirse

Rango y amplitud del movimiento

Miden cuánto se desplazó el precio dentro de una ventana histórica.

Ejemplos observados durante el Stage_06:

```
rolling_range_pct_30m  
rolling_range_pct_60m  
rolling_range_pct_90m  
rolling_range_pts_15m  
rolling_range_pts_30m  
rolling_range_pts_60m
```

Las variables porcentuales expresan el rango en términos relativos.

Las variables en puntos expresan la amplitud absoluta del movimiento, pero no representan directamente el nivel nominal del precio.

Estas features permiten identificar condiciones como:

```
Mercado comprimido  
Mercado expandido  
Ruptura de rango  
Incremento de volatilidad  
Movimiento lateral
```

Volatilidad

Miden cuánto varían los retornos o los precios durante un período reciente.

Ejemplos conceptuales:

```
vol_ret_1m_10m  
vol_ret_1m_30m  
vol_ret_1m_60m  
vol_ret_1m_90m
```

Estas variables permiten evaluar si el mercado se encuentra:

- estable;
- acelerándose;
- presentando movimientos irregulares;
- atravesando un período de alta volatilidad.

La volatilidad es relevante porque los thresholds y las barreras operativas dependen del comportamiento del mercado dentro de cada régimen.

Estadísticas de las barras

Describen el tamaño y la variabilidad de las barras recientes.

Ejemplos observados:

```

bar_range_mean_10m
bar_range_mean_30m
bar_range_mean_60m
bar_range_std_10m
bar_range_std_30m
bar_range_std_60m

```

Estas features resumen el comportamiento reciente de las velas.

Por ejemplo:

```
bar_range_mean_30m
```

indica el tamaño promedio de las barras durante los últimos 30 minutos.

Mientras que:

```
bar_range_std_30m
```

indica si los tamaños de las barras fueron estables o muy variables.

Posición relativa del precio

Representan dónde se encuentra el precio actual respecto de un rango histórico, sin utilizar necesariamente el nivel absoluto.

Ejemplos conceptuales:

```

position_in_range_30m
position_in_range_60m
distance_from_high_pct
distance_from_low_pct

```

Estas variables permiten identificar situaciones como:

```

Precio cerca del máximo reciente
Precio cerca del mínimo reciente
Precio en la zona media del rango
Posible ruptura
Posible rechazo

```

Indicadores técnicos causales

Podrán utilizarse indicadores contruidos únicamente con información histórica.

Ejemplos conceptuales:

```

RSI
ATR normalizado
EMA relativa
ROC
distancia porcentual respecto de una media móvil

```

Estos indicadores solo podrán ingresar si:

- fueron incluidos en el conjunto `reduced_no_level` ;
- utilizan información disponible hasta `t` ;
- fueron aprobados durante la auditoría de leakage.

No se incorporarán nuevos indicadores directamente en esta notebook.

Volumen

Podrán utilizarse variables que describan la actividad reciente del mercado.

Ejemplos conceptuales:

```
volume
relative_volume
rolling_volume_mean
volume_ratio
volume_change
```

Las variables de volumen pueden ayudar a diferenciar:

- movimientos con alta participación;
- movimientos con poca actividad;
- rupturas acompañadas por volumen;
- períodos de baja liquidez.

La lista exacta dependerá de las columnas incluidas en los metadatos del dataset seleccionado.

Contexto intradiario predictivo

Inicialmente se permitirá utilizar:

```
minute_of_day
regime_id
```

Estas variables están disponibles en tiempo real y permiten que el modelo aprenda diferencias entre horarios.

Por ejemplo, una misma condición de volatilidad puede tener significados diferentes durante:

```
Pre-market
Opening
Regular
Closing
```

`minute_of_day` indica la posición temporal dentro de la jornada.

`regime_id` representa el régimen intradiario definido previamente.

Estas variables deberán mantenerse iguales para todos los modelos.

4.5.3. Features que no ingresarán inicialmente

No se utilizarán como features predictivas:

```
year
dataset_split
fold_id
```

target_name
contract_target

Tampoco ingresarán inicialmente:

contract

El contrato se conservará como variable auxiliar para:

- evitar que una ventana cruce un cambio de contrato;
- analizar resultados por contrato;
- detectar posibles diferencias de comportamiento.

La incorporación del contrato como feature requeriría una codificación categórica específica y se considerará únicamente como un experimento posterior.

4.5.4. Representación dentro de una ventana

Cada timestamp de la ventana tendrá el mismo conjunto de features.

Para una ventana de longitud `L`:

`X_sequence.shape = (L, n_features)`

Por ejemplo, con una ventana de 60 minutos y 18 features:

`X_sequence.shape = (60, 18)`

La estructura conceptual será:

Minuto	Feature 1	Feature 2	...	Feature 18
t-59	valor	valor	...	valor
t-58	valor	valor	...	valor
...	...	...	...	...
t	valor	valor	...	valor

El modelo podrá analizar tanto:

- el valor actual de cada feature;
- como su evolución durante los minutos anteriores.

4.5.5. Variables numéricas y categóricas

La mayoría de las features serán numéricas continuas:

retornos
rangos
volatilidades
momentum
estadísticas de barras
volumen

Estas variables podrán requerir escalado.

El escalador deberá ajustarse exclusivamente utilizando los datos de entrenamiento de cada fold.

Conceptualmente:

Train del fold → ajustar escalador

Validation y test → aplicar el mismo escalador

Nunca se deberá ajustar el escalador utilizando información del período de validación o test.

Las variables como `regime_id` no deberán escalarse automáticamente como si fueran una magnitud continua sin analizar previamente su representación.

Su tratamiento definitivo podrá ser:

- codificación one-hot;
- embedding categórico;
- representación numérica controlada.

Para garantizar comparabilidad, se utilizará el mismo tratamiento en todos los modelos.

4.5.6. Obtención de la lista definitiva

La lista de features se obtendrá mediante los metadatos del dataset:

```
FEATURE_COLUMNS = load_feature_columns_from_metadata(  
    dataset_name=DATASET_NAME,  
    feature_set="OPC_reduced_no_level",  
)
```

La función anterior es únicamente una representación conceptual.

La implementación real dependerá del formato en que fueron guardados los metadatos del Stage_05 y Stage_06.

No se utilizará una lista escrita manualmente sin compararla previamente con los metadatos.

4.5.7. Validaciones obligatorias

Antes de aceptar la lista de features se deberá comprobar que:

1. Todas las columnas existen en el dataset.
2. No existen columnas duplicadas.
3. No se incluye el target.
4. No se incluyen otros targets.
5. No se incluyen variables futuras.
6. No se incluyen columnas auxiliares.
7. No se incluye `dataset_split` ni `fold_id`.
8. No existen columnas constantes.

9. No existen valores infinitos.
10. Los valores faltantes están controlados.
11. Todas las features son causales.
12. El orden de las features es idéntico para todos los modelos.
13. La cantidad de features coincide con los metadatos.
14. La selección corresponde realmente a `OPC_reduced_no_level`.

4.5.8. Configuración inicial

La configuración de features será:

```
FEATURE_CONFIG = {
    "feature_set": "OPC_reduced_no_level",
    "include_price_level_features": False,
    "include_minute_of_day": True,
    "include_regime_id": True,
    "include_contract_as_feature": False,
    "include_year_as_feature": False,
    "include_dataset_split_as_feature": False,
    "feature_source": "stage_05_stage_06_metadata",
}
```

La lista definitiva se incorporará después de cargar y validar los metadatos reales:

```
FEATURE_CONFIG["feature_columns"] = FEATURE_COLUMNS
FEATURE_CONFIG["n_features"] = len(FEATURE_COLUMNS)
```

4.5.9. Decisión metodológica

Todos los modelos del experimento principal utilizarán exactamente el mismo conjunto de features:

`OPC_reduced_no_level`

Las diferencias en el desempeño deberán provenir de la arquitectura del modelo y no de cambios en las variables de entrada.

Las versiones con features de nivel, datasets completos u otras variables de contexto se evaluarán únicamente después de establecer una línea base comparable.

4.6. Variables de contexto

Las variables de contexto describen las condiciones temporales y operativas asociadas con cada observación.

No representan directamente el comportamiento del precio, pero permiten identificar en qué situación se produjo cada muestra.

Ejemplos:

```
minute_of_day
regime_id
contract
```



```
year
quarter
trading_date
dataset_split
fold_id
```

Estas variables se dividirán en dos categorías:

Contexto predictivo
→ puede ingresar al modelo.

Contexto analítico
→ se conserva para organizar y analizar resultados,
pero no ingresa al modelo.

La inclusión de una variable de contexto como feature deberá definirse explícitamente.

No todas las variables de contexto podrán utilizarse como entrada.

4.6.1. Contexto predictivo

Las variables de contexto predictivo cumplen las siguientes condiciones:

- están disponibles en el instante de decisión;
- no contienen información futura;
- pueden aportar señal útil;
- no identifican directamente el período de train, validation o test;
- pueden estar disponibles durante una futura operación en vivo.

Para el experimento principal se utilizarán inicialmente:

```
PREDICTIVE_CONTEXT_COLUMNS = [  
    "minute_of_day",  
    "regime_id",  
]
```

Los nombres definitivos deberán verificarse en el dataset y los metadatos, ya que podrían existir variantes como:

```
regime_id_target  
minute_of_day_target
```

4.6.2. minute_of_day

La variable `minute_of_day` indica la posición de cada observación dentro de la jornada intradiaria.

Por ejemplo:

```
04:30 → minuto inicial disponible  
09:30 → apertura del mercado regular  
16:00 → final del período disponible
```

Esta variable permite que el modelo aprenda que una misma condición de mercado puede tener significados diferentes según el horario.

Por ejemplo:

Alta volatilidad durante Opening
no necesariamente representa lo mismo que:

Alta volatilidad durante Regular o Closing
`minute_of_day` está disponible en tiempo real y, por lo tanto, puede utilizarse como feature causal.

No obstante, no se utilizará directamente su valor absoluto sin tratamiento.

Se aplicará una normalización utilizando los límites conocidos de la jornada:

```
minute_of_day_normalized =  
(minute_of_day - minute_min) /  
(minute_max - minute_min)
```

El resultado estará aproximadamente dentro del intervalo:

[0, 1]

Esta transformación facilita el entrenamiento de los modelos neuronales.

Como el período analizado representa una jornada limitada y no un ciclo completo de 24 horas, no será obligatorio utilizar inicialmente una codificación circular mediante seno y coseno.

La posible incorporación de una codificación cíclica podrá evaluarse posteriormente como experimento de robustez.

4.6.3. `regime_id`

La variable `regime_id` identifica el régimen intradiario correspondiente a cada observación.

Los regímenes definidos en los stages anteriores son:

<code>regime_id</code>	Régimen
0	Overnight
1	Pre-market
2	Opening
3	Regular
4	Closing

Esta variable puede aportar información relevante porque:

- la volatilidad cambia según el horario;
- la liquidez no es constante durante la jornada;
- los thresholds del target dependen del régimen;
- la frecuencia de las clases OPC puede variar entre regímenes;
- un mismo patrón de features puede tener distinto significado operativo según el régimen.

Aunque `regime_id` está representado mediante números, sus valores no constituyen una magnitud continua.

Por ejemplo:

```
regime_id = 4
```

no significa que Closing sea el doble de Opening.

Por esta razón, no deberá tratarse como una variable numérica continua ni escalarse como si existiera una distancia matemática entre los regímenes.

Para mantener el mismo tratamiento en todas las arquitecturas se utilizará inicialmente codificación one-hot:

```
regime_overnight  
regime_premarket  
regime_opening  
regime_regular  
regime_closing
```

Ejemplo para una observación del régimen Regular:

```
regime_overnight = 0  
regime_premarket = 0  
regime_opening = 0  
regime_regular = 1  
regime_closing = 0
```

La utilización de embeddings para representar el régimen podrá evaluarse posteriormente, pero no se utilizará en la comparación inicial porque introduciría diferencias adicionales entre arquitecturas.

4.6.4. Relación entre `minute_of_day` y `regime_id`

Ambas variables describen el horario, pero no contienen exactamente la misma información.

```
minute_of_day  
→ indica el minuto exacto de la jornada.
```

```
regime_id  
→ indica el bloque operativo al que pertenece ese minuto.
```

Por ejemplo:

```
10:35 y 14:40
```

pertenecen al mismo régimen Regular, pero corresponden a momentos diferentes de la jornada.

Por esta razón, inicialmente se conservarán ambas variables.

Posteriormente se podrá estudiar si una de ellas resulta redundante o si su combinación mejora la capacidad predictiva.

4.6.5. Contexto analítico

Las variables de contexto analítico se conservarán asociadas con cada muestra, pero no ingresarán al modelo principal.

La configuración inicial será:

```
ANALYTICAL_CONTEXT_COLUMNS = [  
    "trading_date",  
    "year",  
    "quarter",  
    "contract",  
]
```

Estas variables permitirán analizar los resultados por:

- día;
- año;
- trimestre;
- contrato;
- régimen;
- período walk-forward.

4.6.6. trading_date

La variable `trading_date` identifica la jornada a la que pertenece cada observación.

Se utilizará para:

- evitar que las ventanas crucen días;
- separar train y validation mediante jornadas completas;
- ordenar las muestras cronológicamente;
- analizar resultados diarios;
- reconstruir posteriormente las predicciones.

No ingresará al modelo porque actúa como identificador temporal.

4.6.7. year

La variable `year` permitirá analizar si el rendimiento del modelo cambia entre años.

Se utilizará para:

- construir los folds walk-forward;
- comparar desempeño temporal;
- detectar degradación del modelo;
- identificar posibles cambios estructurales.

No ingresará al modelo porque podría permitir que la red identifique directamente el período histórico.

Ejemplo:

```
year = 2020  
year = 2021  
year = 2022
```

Esto podría facilitar que el modelo memorice diferencias entre años en lugar de aprender patrones generales del mercado.

4.6.8. quarter

La variable `quarter` se conservará para realizar análisis trimestrales.

Podrá utilizarse para estudiar:

- cambios estacionales;
- estabilidad dentro de cada año;
- concentración de errores;
- posibles diferencias entre períodos contractuales.

No ingresará inicialmente al modelo.

Su inclusión futura requerirá demostrar que aporta información causal y generalizable sin actuar simplemente como identificador temporal.

4.6.9. contract

La variable `contract` identifica el contrato de futuros correspondiente a cada observación.

Ejemplos:

```
H20  
M20  
U20  
Z20
```

Se conservará como variable auxiliar y analítica para:

- impedir que una ventana cruce cambios de contrato;
- analizar resultados por contrato;
- identificar posibles problemas durante los rollovers;
- comprobar estabilidad entre contratos.

No ingresará inicialmente al modelo porque el código del contrato contiene información temporal.

Por ejemplo:

```
H20
```

permite identificar indirectamente el año 2020.

Esto podría facilitar la memorización de períodos históricos y reducir la capacidad de generalización.

4.6.10. Variables de control del experimento

Las siguientes variables podrán ser necesarias para organizar los experimentos:

```
dataset_split
fold_id
validation_year
target_name
dataset_name
```

Estas variables nunca ingresarán al modelo.

Se utilizarán únicamente para:

- identificar train, validation y test;
- construir los folds;
- guardar los resultados;
- comprobar la trazabilidad;
- consolidar métricas entre experimentos.

Permitir que estas columnas ingresen al modelo produciría leakage directo sobre la estructura temporal del experimento.

4.6.11. Contexto dentro de las ventanas

Las variables predictivas de contexto se incorporarán en cada minuto de la secuencia.

Para una ventana de 60 minutos:

```
t-59 → minute_of_day + régimen
t-58 → minute_of_day + régimen
...
t    → minute_of_day + régimen
```

Esto permite que el modelo observe posibles cambios dentro de la ventana.

Por ejemplo, una secuencia podría comenzar en Pre-market y finalizar en Opening:

```
09:10-09:29 → Pre-market
09:30-10:09 → Opening
```

La ventana podrá contener más de un régimen siempre que:

- pertenezca al mismo día;
- no presente huecos temporales;
- no cruce un cambio de contrato.

No se impedirá que una ventana cruce de un régimen intradiario a otro, porque esa transición puede contener información relevante.

4.6.12. Separación de las variables

La configuración general será:

```
PREDICTIVE_CONTEXT_COLUMNS = [  
    "minute_of_day",  
    "regime_id",  
]  
  
ANALYTICAL_CONTEXT_COLUMNS = [  
    "trading_date",  
    "year",  
    "quarter",  
    "contract",  
]  
  
EXPERIMENT_CONTROL_COLUMNS = [  
    "dataset_split",  
    "fold_id",  
    "validation_year",  
    "target_name",  
    "dataset_name",  
]
```

La lista exacta deberá adaptarse a los nombres reales presentes en el dataset.

4.6.13. Tratamiento inicial

El tratamiento definido será:

Variable	Ingresa al modelo	Tratamiento
minute_of_day	Sí	Normalización entre 0 y 1
regime_id	Sí	Codificación one-hot
trading_date	No	Auxiliar
year	No	Analítica y construcción de folds
quarter	No	Analítica
contract	No	Auxiliar y analítica
dataset_split	No	Control experimental
fold_id	No	Control experimental

Todos los modelos deberán utilizar exactamente la misma representación de estas variables.

4.6.14. Validaciones obligatorias

Antes de construir las ventanas se deberá comprobar que:

1. `minute_of_day` existe y presenta valores válidos.
2. `regime_id` contiene únicamente los regímenes esperados.
3. Cada minuto pertenece a un único régimen.
4. La relación entre `minute_of_day` y `regime_id` es consistente.
5. `trading_date` coincide con la jornada de cada timestamp.
6. Ninguna ventana cruza días.
7. Ninguna ventana cruza cambios de contrato.
8. `year` coincide con el timestamp.
9. `quarter` coincide con el timestamp.
10. `dataset_split` y `fold_id` no ingresan al modelo.
11. La codificación del régimen es idéntica en todos los folds.
12. Las categorías one-hot mantienen siempre el mismo orden.
13. Las variables de contexto no contienen valores futuros.
14. Las columnas utilizadas coinciden con los metadatos.

4.6.15. Configuración inicial

La configuración correspondiente será:

```
CONTEXT_CONFIG = {
    "predictive_context": {
        "minute_of_day": {
            "include": True,
            "encoding": "min_max_normalization",
        },
        "regime_id": {
            "include": True,
            "encoding": "one_hot",
            "categories": [0, 1, 2, 3, 4],
        },
    },
    "analytical_context": [
        "trading_date",
        "year",
        "quarter",
        "contract",
    ],
    "experiment_control": [
        "dataset_split",
        "fold_id",
        "validation_year",
        "target_name",
        "dataset_name",
    ],
    "include_contract_as_feature": False,
    "include_year_as_feature": False,
    "include_quarter_as_feature": False,
}
```

4.6.16. Decisión metodológica

Para el experimento principal se utilizarán como contexto predictivo:

```
minute_of_day  
regime_id
```

Las variables:

```
trading_date  
year  
quarter  
contract  
dataset_split  
fold_id
```

se conservarán para validar, organizar y analizar los experimentos, pero no ingresarán al modelo.

De esta manera, la red podrá conocer el momento intradiario de cada observación sin recibir identificadores que le permitan memorizar directamente el período histórico.

4.7. Columnas auxiliares y excluidas

Antes de construir las ventanas temporales, se deberán separar explícitamente las columnas auxiliares y las columnas excluidas.

La diferencia principal será:

Columnas auxiliares

→ se conservan para organizar, validar y analizar el experimento.

Columnas excluidas

→ no se utilizan como entrada del modelo y, cuando no sean necesarias,

tampoco se conservarán en el dataset secuencial.

En ningún caso estas columnas deberán formar parte de la matriz predictiva **X**.

4.7.1. Columnas auxiliares

Las columnas auxiliares contienen información necesaria para mantener la trazabilidad temporal y estructural de cada muestra.

Podrán utilizarse para:

- ordenar cronológicamente los datos;
- identificar días de trading;
- evitar que las ventanas crucen jornadas;
- detectar cambios de contrato;
- construir los folds walk-forward;
- asociar cada predicción con su timestamp;
- analizar resultados por año, régimen o contrato;
- reconstruir posteriormente las operaciones;
- guardar predicciones y probabilidades.

La configuración inicial será:

```
AUXILIARY_COLUMNS = [  
    "datetime",  
    "trading_date",  
    "year",  
    "quarter",  
    "contract",  
    "dataset_split",  
    "fold_id",  
]
```

La lista exacta deberá adaptarse a los nombres reales presentes en el dataset.

4.7.2. datetime

El índice temporal o la columna `datetime` identifica el instante exacto de cada observación.

Se utilizará para:

- mantener el orden cronológico;
- verificar continuidad temporal;
- alinear features y target;
- construir las ventanas;
- guardar las predicciones fuera de muestra;
- reconstruir posteriormente el momento de entrada.

No ingresará al modelo como valor numérico.

El timestamp completo contiene información temporal absoluta que podría permitir identificar indirectamente el período histórico.

4.7.3. trading_date

La variable `trading_date` identifica la jornada de trading correspondiente a cada observación.

Se utilizará para:

- impedir que una ventana cruce días;
- separar train y validation mediante jornadas completas;
- comprobar que el target tampoco cruce jornadas;
- analizar métricas por día.

No ingresará al modelo.

4.7.4. contract

La variable `contract` identifica el contrato del MNQ asociado con cada observación.

Se utilizará para:

- impedir que una ventana cruce un cambio de contrato;
- analizar resultados por contrato;
- identificar posibles problemas durante el rollover;
- verificar la continuidad de las series.

No ingresará inicialmente al modelo porque el código del contrato contiene información temporal.

4.7.5. Variables de control experimental

Las siguientes columnas podrán utilizarse para organizar los experimentos:

```
dataset_split  
fold_id  
validation_year  
dataset_name  
feature_set  
target_name
```

Estas variables permiten identificar:

- el dataset utilizado;
- el fold walk-forward;
- el período de train, validation o test;
- el target;
- el conjunto de features;
- el origen de cada resultado.

Nunca deberán ingresar al modelo porque revelan directamente la estructura del experimento.

4.7.6. Columnas excluidas por leakage

Se excluirán todas las variables que contengan o representen información futura.

Ejemplos conceptuales:

```
future_high_max  
future_low_min  
future_close  
future_return  
up_excursion_pts  
down_excursion_pts  
mfe_pts  
mae_pts  
tp_hit  
sl_hit  
first_barrier_hit  
bars_to_event  
trade_result
```

Estas columnas pueden ser útiles para construir o analizar el target, pero no pueden utilizarse como features.

La regla será:

Si una variable necesita observar barras posteriores a t , no puede ingresar al modelo.

4.7.7. Otros targets

También se excluirán todas las columnas correspondientes a otros targets.

Ejemplos:

```
dir_p40_h30
dir_p50_h60
bar_p50_h60_tp15_sl10
otros targets OPC
```

El único target utilizado como salida será:

```
TARGET_COLUMN = "opc_p50_h60_tp15_sl10"
```

Ningún target podrá formar parte de la matriz de entrada.

Esto incluye tanto el target principal como otros targets relacionados.

4.7.8. Columnas utilizadas para construir el target

Se excluirán las variables intermedias utilizadas durante la construcción de DIR, BAR y OPC.

Ejemplos conceptuales:

```
threshold_pts
tp_distance_pts
sl_distance_pts
tp_price
sl_price
direction_label
barrier_label
opc_label_text
```

Aunque algunas de estas variables puedan estar disponibles en el instante t , podrían contener una representación directa de la lógica utilizada para construir la etiqueta.

Su inclusión deberá evitarse inicialmente para impedir que el modelo aprenda una versión simplificada o indirecta del target.

Las variables necesarias para interpretar o reconstruir posteriormente una operación podrán conservarse como auxiliares, pero separadas de X .

4.7.9. Columnas constantes

Las columnas constantes no aportan información predictiva.

Ejemplo:

`feature_x = 1` para todas las observaciones

Estas columnas:

- no ayudan al modelo;
- aumentan innecesariamente la dimensionalidad;
- consumen memoria;
- pueden producir problemas durante el escalado.

Por lo tanto, deberán eliminarse antes de construir las ventanas.

4.7.10. Columnas duplicadas

Se comprobará si existen columnas con valores idénticos.

Por ejemplo:

```
feature_a == feature_b
```

Si dos columnas contienen exactamente la misma información, se conservará solamente una.

También se comprobarán:

- nombres duplicados;
- columnas repetidas por procesos de merge;
- variables equivalentes con distinto sufijo.

4.7.11. Identificadores sin valor predictivo

Se excluirán identificadores que no representen condiciones reales del mercado.

Ejemplos:

```
row_id  
sample_id  
dataset_id  
experiment_id  
source_file
```

Estas columnas pueden utilizarse para trazabilidad, pero no contienen una relación causal con el resultado futuro.

4.7.12. Columnas de split temporal

Las columnas que indican directamente si una observación pertenece a train, validation o test deberán excluirse.

Ejemplos:

```
dataset_split  
split_name
```

```

fold_id
is_train
is_validation
is_test
validation_year

```

Si estas variables ingresaran al modelo, la red podría identificar directamente el período histórico evaluado.

Esto constituiría leakage experimental.

4.7.13. Separación entre auxiliares y excluidas

La clasificación general será:

```

AUXILIARY_COLUMNS = [
    "datetime",
    "trading_date",
    "year",
    "quarter",
    "contract",
    "dataset_split",
    "fold_id",
]
EXCLUDED_COLUMN_PATTERNS = [
    "future_",
    "mfe",
    "mae",
    "tp_hit",
    "sl_hit",
    "barrier",
    "target",
    "split",
    "fold",
]

```

Esta lista de patrones es únicamente una referencia conceptual.

No se deberán excluir columnas automáticamente basándose solamente en su nombre sin revisar previamente su función.

Por ejemplo, una feature podría contener la palabra `range` o `target_distance` sin implicar necesariamente leakage.

La exclusión definitiva deberá combinar:

- metadatos del Stage_05 y Stage_06;
- revisión de nombres;
- revisión conceptual;
- validación directa del cálculo de cada columna.

4.7.14. Matrices finales

La separación del dataset se realizará conceptualmente de la siguiente manera:

```
X = df[FEATURE_COLUMNS]
```

```
y = df[TARGET_COLUMN]
```

```
metadata = df[AUXILIARY_COLUMNS]
```

Las columnas excluidas no deberán aparecer en `X` :

```
assert not set(EXCLUDED_COLUMNS).intersection(FEATURE_COLUMNS)
```

También se verificará:

```
assert TARGET_COLUMN not in FEATURE_COLUMNS
```

y:

```
assert not set(AUXILIARY_COLUMNS).intersection(FEATURE_COLUMNS)
```

excepto en los casos definidos explícitamente como contexto predictivo, como:

```
minute_of_day
regime_id
```

4.7.15. Validaciones obligatorias

Antes de construir las ventanas se deberá comprobar que:

1. El target no está incluido entre las features.
2. Ningún otro target está incluido en `FEATURE_COLUMNS`.
3. No existen variables futuras.
4. No existen columnas asociadas con MFE o MAE.
5. No existen resultados de TP o SL dentro de `X`.
6. No existen columnas de train, validation o test dentro de `X`.
7. No existen identificadores del fold dentro de `X`.
8. No existen columnas constantes.
9. No existen columnas duplicadas.
10. Las columnas auxiliares están separadas de la matriz predictiva.
11. Las columnas necesarias para construir las ventanas están disponibles.
12. El contrato y la fecha permiten detectar cambios de grupo.
13. Las predicciones podrán asociarse nuevamente con su timestamp.
14. La lista final coincide con los metadatos de los stages anteriores.

4.7.16. Configuración inicial

La configuración correspondiente será:

```
COLUMN_ROLE_CONFIG = {
    "target_column": "opc_p50_h60_tp15_sl10",
    "auxiliary_columns": [
        "datetime",
        "trading_date",
        "year",
        "quarter",
        "contract",
        "dataset_split",
```

```

        "fold_id",
    ],
    "predictive_context_columns": [
        "minute_of_day",
        "regime_id",
    ],
    "exclude_other_targets": True,
    "exclude_future_columns": True,
    "exclude_split_columns": True,
    "exclude_constant_columns": True,
    "exclude_duplicate_columns": True,
}

```

La lista definitiva se completará después de cargar el dataset y revisar sus columnas reales.

4.7.17. Decisión metodológica

Las columnas auxiliares se conservarán para garantizar la trazabilidad y la correcta construcción de las ventanas.

Las columnas excluidas no ingresarán al modelo porque:

- contienen información futura;
- representan directamente el resultado;
- identifican el split temporal;
- no aportan señal predictiva;
- pueden introducir leakage;
- aumentan innecesariamente la dimensionalidad.

La regla principal será:

Toda columna debe tener una función claramente definida.

Si no puede justificarse por qué una columna debe ingresar al modelo,
se mantendrá fuera de la matriz predictiva hasta completar su revisión.

4.8. Validaciones obligatorias del dataset experimental

Antes de generar las ventanas temporales se realizará una validación final del dataset seleccionado.

El objetivo será confirmar que los datos son consistentes, causales y aptos para ser utilizados por todos los modelos del Stage_07.

4.8.1. Validación estructural

Se comprobará que:

- el índice sea `datetime` ;
- el índice esté ordenado cronológicamente;
- no existan timestamps duplicados;
- el target esté presente;
- todas las features esperadas existan;
- la lista de features coincida con los metadatos;
- no existan columnas duplicadas o constantes.

4.8.2. Validación de calidad

Para las columnas utilizadas se verificará:

- ausencia de valores infinitos;
- control de valores faltantes;
- tipos de datos correctos;
- valores válidos de `regime_id` ;
- valores válidos de `minute_of_day` ;
- presencia de las cinco clases OPC.

Los valores faltantes no se completarán globalmente antes de dividir los folds.

Si fuera necesaria una imputación, deberá ajustarse únicamente con los datos de entrenamiento de cada fold.

4.8.3. Validación de leakage

Se confirmará que `FEATURE_COLUMNS` no contenga:

- el target principal;
- otros targets;
- información futura;
- MFE o MAE;
- resultados de TP o SL;
- variables utilizadas para construir la etiqueta;
- identificadores de train, validation o test;
- `fold_id` , `dataset_split` o `validation_year` .

La regla será:

Toda feature debe estar disponible en el instante t .

4.8.4. Validación temporal

Se comprobará que:

- cada timestamp pertenezca a un único día de trading;
- cada observación tenga contrato y régimen válidos;
- no existan huecos temporales inesperados;

- las ventanas puedan construirse sin cruzar días;
- las ventanas no crucen cambios de contrato;
- el target esté alineado con el último instante de cada ventana.

Las observaciones sin suficiente historial para completar una ventana serán descartadas.

4.8.5. Resultado de la validación

La validación deberá producir un resumen como el siguiente:

```
Dataset válido: Sí / No
Número de observaciones
Número de features
Número de clases
NaN detectados
Inf detectados
Duplicados
Columnas constantes
Posible leakage
Rango temporal
Estado final
```

El dataset solo podrá pasar a la notebook de generación de ventanas si el estado final es:

APROBADO

En caso contrario, se deberá corregir la causa antes de continuar.

4.9. Configuración inicial del dataset experimental

La configuración principal del dataset para el Stage_07 será:

```
DATASET_CONFIG = {
    # Dataset principal
    "dataset_name": (
        "opc_p50_h60_tp15_sl10"
        "__OPC_reduced_no_level"
        "__all_regimes"
    ),

    # Target
    "target_column": "opc_p50_h60_tp15_sl10",

    # Conjunto de features
    "feature_set": "OPC_reduced_no_level",
    "include_price_level_features": False,

    # Contexto predictivo
    "predictive_context_columns": [
        "minute_of_day",
        "regime_id",
    ],

    # Contexto analítico y auxiliar
    "auxiliary_columns": [
```

```

        "datetime",
        "trading_date",
        "year",
        "quarter",
        "contract",
        "dataset_split",
        "fold_id",
    ],

    # Variables que no ingresan al modelo
    "include_contract_as_feature": False,
    "include_year_as_feature": False,
    "include_quarter_as_feature": False,
    "include_split_columns_as_features": False,

    # Reglas de exclusión
    "exclude_other_targets": True,
    "exclude_future_columns": True,
    "exclude_constant_columns": True,
    "exclude_duplicate_columns": True,

    # Reglas temporales
    "allow_cross_day_windows": False,
    "allow_cross_contract_windows": False,
    "allow_cross_regime_windows": True,

    # Fuente de la selección
    "feature_source": "stage_05_stage_06_metadata",
}

```

La lista definitiva de features no se escribirá manualmente.

Se obtendrá directamente de los metadatos del dataset seleccionado:

```

FEATURE_COLUMNS = load_feature_columns_from_metadata(
    dataset_name=DATASET_CONFIG["dataset_name"],
    feature_set=DATASET_CONFIG["feature_set"],
)

```

La implementación exacta de esta función dependerá del formato real de los metadatos guardados en los stages anteriores.

Una vez cargadas las columnas, se completará la configuración:

```

DATASET_CONFIG["feature_columns"] = FEATURE_COLUMNS
DATASET_CONFIG["n_features"] = len(FEATURE_COLUMNS)

```

También se comprobará que las variables de contexto predictivo estén incluidas una sola vez:

```

MODEL_INPUT_COLUMNS = list(
    dict.fromkeys(
        FEATURE_COLUMNS
        + DATASET_CONFIG["predictive_context_columns"]
    )
)

```

Finalmente:

```
DATASET_CONFIG["model_input_columns"] = MODEL_INPUT_COLUMNS
DATASET_CONFIG["n_model_inputs"] = len(MODEL_INPUT_COLUMNS)
```

Antes de aprobar la configuración se deberá verificar:

```
assert DATASET_CONFIG["target_column"] not in MODEL_INPUT_COLUMNS

assert not set(
    DATASET_CONFIG["auxiliary_columns"]
).intersection(MODEL_INPUT_COLUMNS)

assert len(MODEL_INPUT_COLUMNS) == len(set(MODEL_INPUT_COLUMNS))
```

La configuración quedará aprobada únicamente si el dataset supera las validaciones definidas en el Punto 4.8.

El resultado esperado será:

```
Dataset experimental definido
Features identificadas
Target separado
Contexto predictivo definido
Columnas auxiliares separadas
Columnas con leakage excluidas
Configuración lista para construir ventanas
```

Esta configuración será utilizada por todas las notebooks posteriores para asegurar que los modelos reciban exactamente los mismos datos.

5. Definición de las ventanas temporales

5.1. Objetivo

Los datasets actuales tienen una estructura tabular:

Un timestamp → una fila de features → un target

En el Stage_07 queremos que el modelo no observe solamente el estado del mercado en el instante `t`, sino también cómo evolucionaron las features durante los minutos anteriores.

La nueva estructura será:

Secuencia histórica que termina en `t` → target OPC correspondiente a `t`

La generación física de estas ventanas se realizará posteriormente en:

`S07_01_sequence_dataset.ipynb`

En esta notebook se definirán únicamente las reglas que deberá seguir ese proceso.

5.2. Definición de una ventana

Para una ventana de longitud `L`, asociada con un timestamp objetivo `t`, la entrada estará formada por:

$t-L+1, \dots, t-2, t-1, t$

El target será el correspondiente al último timestamp de la ventana:

$[X_{t-L+1}, \dots, X_{t-1}, X_t] \rightarrow y_t$

Por ejemplo, para una ventana de 60 minutos:

Timestamp objetivo: 10:30

Entrada:

09:31, 09:32, ..., 10:29, 10:30

Target:

clase OPC correspondiente a las 10:30

La ventana contiene exactamente 60 observaciones:

Desde $t-59$ hasta t

El horizonte futuro utilizado para construir el target comienza después:

$t+1, t+2, \dots, t+60$

Por lo tanto:

Ventana histórica:

información que recibe el modelo.

Horizonte futuro:

información utilizada solamente para construir la respuesta correcta.

El modelo nunca recibe las barras posteriores a t .

5.3. Diferencia entre lookback y horizonte del target

No se deben confundir estos dos conceptos:

Lookback:

cantidad de minutos históricos que observa el modelo.

Horizonte del target:

cantidad máxima de minutos futuros utilizada para definir la clase OPC.

Para el experimento principal:

Lookback principal = 60 minutos

Horizonte del target = 60 minutos

Ambos tienen inicialmente la misma duración, pero representan cosas diferentes.

Podríamos utilizar, por ejemplo:

Lookback = 30 minutos

Horizonte del target = 60 minutos

En ese caso, el modelo observaría 30 minutos históricos para predecir un resultado definido sobre los siguientes 60 minutos.

5.4. Ventanas candidatas

Las longitudes inicialmente consideradas serán:

```
LOOKBACK_WINDOWS = [30, 60, 90]
```

Estas ventanas permitirán comparar diferentes cantidades de información histórica:

Lookback	Interpretación
30 minutos	Contexto reciente
60 minutos	Contexto intermedio
90 minutos	Contexto más amplio

La primera comparación entre arquitecturas utilizará:

```
PRIMARY_LOOKBACK = 60
```

Esto significa que inicialmente todos los modelos recibirán exactamente los mismos 60 minutos históricos.

Las ventanas de 30 y 90 minutos se evaluarán posteriormente sobre los modelos que presenten mejores resultados.

5.5. Forma de los datos

La representación principal tendrá tres dimensiones:

```
X.shape = (  
    número de muestras,  
    longitud de la ventana,  
    número de features  
)
```

Ejemplo:

```
X.shape = (500000, 60, 18)
```

Esto significa:

500000 muestras
60 minutos por muestra
18 features por minuto

Cada muestra tendrá una estructura similar a:

Minuto	Feature 1	Feature 2	...	Feature 18
t-59	valor	valor	...	valor
t-58	valor	valor	...	valor
...	...	...	...	...
t	valor	valor	...	valor

El target tendrá una dimensión:

```
y.shape = (número de muestras,)
```

Ejemplo:

```
y.shape = (500000,)
```

También se conservará información auxiliar para cada ventana:

```
timestamp final  
trading_date  
contract  
regime_id final  
fold  
target
```

5.6. Reglas de construcción

Una ventana será válida únicamente si cumple todas las reglas siguientes.

Información causal

La ventana deberá:

- terminar exactamente en `t` ;
- utilizar únicamente información disponible hasta `t` ;
- no contener barras posteriores a `t` ;
- asociarse con el target correspondiente a `t` .

Día de trading

La ventana completa deberá pertenecer al mismo día.

Ejemplo válido:

```
09:31-10:30 del mismo día
```

Ejemplo inválido:

```
15:31 del día anterior-04:30 del día siguiente
```

No se utilizarán datos del día anterior para completar las primeras ventanas de una nueva jornada.

Continuidad temporal

Las observaciones deberán ser consecutivas según la frecuencia esperada de un minuto.

Ejemplo válido:

```
10:00, 10:01, 10:02, 10:03
```

Ejemplo inválido:

```
10:00, 10:01, 10:05, 10:06
```

Si existe un hueco inesperado dentro de la secuencia, la ventana se descartará.

Contrato

Toda la ventana deberá pertenecer al mismo contrato.

Ejemplo válido:

Todos los minutos pertenecen a M23

Ejemplo inválido:

Parte de la ventana pertenece a H23
y otra parte pertenece a M23

Esto evita combinar información de contratos diferentes dentro de una misma muestra.

Régimen intradiario

Se permitirá que una ventana cruce de un régimen a otro, siempre que permanezca dentro del mismo día.

Ejemplo válido:

09:10-09:29 → Pre-market

09:30-10:09 → Opening

Esta transición puede contener información útil para el modelo.

5.7. Observaciones sin historia suficiente

Al comienzo de cada jornada no existen suficientes minutos anteriores para completar todas las ventanas.

Por ejemplo, para una ventana de 60 minutos:

Primer minuto disponible del día:
no tiene 59 minutos anteriores.

Minuto número 30:
todavía no tiene una ventana completa.

Minuto número 60:
primera ventana completa posible.

Por lo tanto, las primeras **L-1** observaciones de cada día no podrán utilizarse como final de una ventana de longitud **L**.

Para:

$$L = 60$$

se perderán como máximo:

59 posibles timestamps objetivo al inicio de cada jornada

Estas observaciones no se descartan por errores de calidad, sino porque no tienen suficiente historial dentro del mismo día.

La cantidad exacta de ventanas válidas se calculará después de aplicar también los controles de continuidad y contrato.

5.8. Representación común para todos los modelos

Se generará una única representación secuencial principal:

`(n_samples, lookback, n_features)`

Esta misma información será utilizada por todas las arquitecturas.

Los modelos secuenciales recibirán directamente la estructura 3D:

CNN 1D
LSTM
GRU
TCN

Para el MLP, cada ventana podrá transformarse temporalmente en una fila:

`(n_samples, lookback × n_features)`

Ejemplo:

Ventana original:
`(60, 18)`

Entrada a MLP:
`(1080,)`

No se generará un dataset diferente para cada modelo. La misma ventana será adaptada según la arquitectura.

5.9. Preprocesamiento y ventanas

Las ventanas deberán generarse conservando la información original y su alineación temporal.

No se aplicará un escalado global utilizando todo el dataset.

Para cada fold:

Train del fold:
ajusta el escalador.

Validation y test:
utilizan el escalador aprendido con train.

Esto evita que la distribución de períodos futuros influya en la transformación de los datos de entrenamiento.

El orden de las features deberá ser exactamente el mismo en:

Train
Validation interna
Validación walk-forward
Test final

5.10. Validaciones de las ventanas

Después de generarlas se comprobará que:

- todas tengan exactamente la longitud esperada;
- todas tengan el mismo número y orden de features;
- ninguna cruce días;
- ninguna cruce cambios de contrato;
- ninguna contenga huecos temporales;
- el último timestamp coincida con el timestamp del target;
- no existan datos posteriores a `t` ;
- no existan targets faltantes;
- las ventanas estén ordenadas cronológicamente;
- no existan ventanas duplicadas.

También se generará un resumen por lookback:

```
Lookback
Número de ventanas válidas
Número de ventanas descartadas
Descartadas por falta de historial
Descartadas por huecos
Descartadas por cambio de contrato
Rango temporal
Forma final de X
Forma final de y
```

5.11. Configuración inicial

```
WINDOW_CONFIG = {
    "candidate_lookbacks": [30, 60, 90],
    "primary_lookback": 60,
    "frequency_minutes": 1,
    "include_current_timestamp": True,
    "target_aligned_to_window_end": True,
    "allow_cross_day": False,
    "allow_time_gaps": False,
    "allow_cross_contract": False,
    "allow_cross_regime": True,
    "expected_input_dimensions": 3,
    "input_dtype": "float32",
}
```

Para el experimento principal:

```
PRIMARY_WINDOW_CONFIG = {
    **WINDOW_CONFIG,
    "lookback": WINDOW_CONFIG["primary_lookback"],
}
```

5.12. Decisión metodológica

La primera comparación entre modelos utilizará ventanas de 60 minutos:

Features desde $t-59$ hasta t
 → target OPC correspondiente a t

Las ventanas:

- serán causales;
- pertenecerán a un único día;
- pertenecerán a un único contrato;
- no contendrán huecos;
- podrán cruzar regímenes intradiarios;
- tendrán exactamente el mismo contenido para todos los modelos.

Las ventanas de 30 y 90 minutos se evaluarán posteriormente para estudiar si los mejores modelos se benefician de observar una historia más corta o más extensa.

6. Esquema de validación walk-forward

Mantendremos los folds del Stage_06:

WF_01
 Train: 2020–2021
 Validation: 2022

WF_02
 Train: 2020–2022
 Validation: 2023

WF_03
 Train: 2020–2023
 Validation: 2024

En cada fold, el período de **Validation** representa la evaluación fuera de muestra. Es decir, el modelo deberá ser entrenado únicamente con los años anteriores y luego evaluado sobre el año siguiente.

Sin embargo, las redes neuronales necesitan decidir cuántas épocas deben entrenarse antes de comenzar a sobreajustar. Para resolverlo, el período de train de cada fold se dividirá temporalmente en dos partes:

```

Train general del fold
├── Train interno
│   └── el modelo aprende y actualiza sus pesos
└── Validation interna
    └── controla el early stopping y determina la mejor época

Validation walk-forward
└── evalúa el modelo terminado sobre datos posteriores
  
```

Por ejemplo, para WF_01 :

Train interno:
2020 – septiembre de 2021

Validation interna:
octubre – diciembre de 2021

Validation walk-forward:
todo 2022

Durante la primera etapa, el modelo se entrena con el train interno y se observa su desempeño sobre la validation interna después de cada época.

Ejemplo:

Época 10 → Macro F1 = 0.38
Época 11 → Macro F1 = 0.40
Época 12 → Macro F1 = 0.42
Época 13 → Macro F1 = 0.41

En este caso, la mejor cantidad de épocas sería:

`best_epoch = 12`

Luego se crea un modelo nuevo desde cero y se entrena utilizando todo el train general del fold:

`2020–2021`

durante exactamente 12 épocas.

Finalmente, el modelo se evalúa una sola vez sobre:

`2022`

El año de validación walk-forward no debe utilizarse para decidir cuándo detener el entrenamiento, porque dejaría de ser una evaluación completamente nueva.

La lógica general será:

Train interno
→ aprende los pesos.

Validation interna
→ decide cuándo detener el entrenamiento.

Train general completo
→ entrena nuevamente el modelo durante la cantidad de épocas seleccionada.

Validation walk-forward
→ evalúa el modelo terminado fuera de muestra.

En resumen:

La validation interna ayuda a construir el modelo.

La validation walk-forward evalúa el modelo final.

6.1. Objetivo

El Stage_07 mantendrá el esquema walk-forward definido y validado en el Stage_06.

Su objetivo es simular una situación real:

- Entrenar con información histórica
 - evaluar sobre un período posterior no utilizado durante el entrenamiento

Los datos nunca se mezclarán aleatoriamente, porque se debe respetar su orden temporal.

6.2. Folds principales

Se utilizarán tres folds con ventana de entrenamiento expansiva:

Fold	Train general	Validación walk-forward
WF_01	2020–2021	2022
WF_02	2020–2022	2023
WF_03	2020–2023	2024

La ventana de entrenamiento crece en cada fold:

- WF_01 → aprende con 2020–2021
- WF_02 → incorpora también 2022
- WF_03 → incorpora también 2023

La validación siempre corresponde al año inmediatamente posterior.

6.3. División interna del train

Las redes neuronales necesitan una validación interna para controlar el entrenamiento.

Por esta razón, el train general de cada fold se dividirá temporalmente en:

- Train interno
 - ajusta los pesos del modelo.

- Validation interna
 - controla el early stopping y selecciona la mejor época.

La estructura será:

- Train general del fold
 - └ Train interno
 - └ Validation interna
- Validación walk-forward
 - └ Evaluación fuera de muestra

La validación walk-forward nunca se utilizará para decidir cuándo detener el entrenamiento.

6.4. División temporal propuesta

Inicialmente, la validación interna utilizará aproximadamente el último trimestre del train general.

WF_01

Train interno:
2020 – septiembre de 2021

Validation interna:
octubre – diciembre de 2021

Validación walk-forward:
2022

WF_02

Train interno:
2020 – septiembre de 2022

Validation interna:
octubre – diciembre de 2022

Validación walk-forward:
2023

WF_03

Train interno:
2020 – septiembre de 2023

Validation interna:
octubre – diciembre de 2023

Validación walk-forward:
2024

Las fechas exactas se ajustarán al último día de trading disponible de cada período.

Las divisiones siempre utilizarán días completos. Una jornada no podrá quedar parcialmente en train y parcialmente en validation.

6.5. Entrenamiento en dos etapas

Cada fold se entrenará en dos etapas.

Etapas 1: determinar la cantidad de épocas

El modelo se entrenará utilizando:

Train interno

→ entrenamiento del modelo.

Validation interna

→ seguimiento de la métrica y early stopping.

Ejemplo:

El mejor resultado de validation interna se obtiene en la época 14.

La mejor época se guardará como:

`best_epoch = 14`

Etapa 2: entrenamiento definitivo del fold

Después se creará un modelo nuevo desde cero.

Este modelo se entrenará utilizando todo el train general:

Train interno + validation interna

Durante:

`best_epoch`

épocas.

Finalmente, se evaluará sobre el año de validación walk-forward.

Ejemplo para `WF_01` :

Desarrollo:

2020-septiembre de 2021 → train interno

octubre-diciembre de 2021 → validation interna

Mejor época:

14

Entrenamiento definitivo:

todo 2020-2021 durante 14 épocas

Evaluación fuera de muestra:

2022

Así se aprovechan todos los datos disponibles del train general sin utilizar el año de validación walk-forward para controlar el entrenamiento.

6.6. Preprocesamiento dentro de cada fold

Todo preprocesamiento deberá aprenderse exclusivamente con los datos permitidos en cada etapa.

Durante la primera etapa:

Train interno

→ ajustar escalador, imputador y pesos de clase.

Validation interna

→ aplicar las transformaciones aprendidas con train interno.

Durante el entrenamiento definitivo:

Train general completo

→ volver a ajustar las transformaciones.

Validación walk-forward

→ aplicar las transformaciones aprendidas con el train general.

Nunca se ajustarán transformaciones utilizando datos del año de validación walk-forward.

6.7. Uso de los resultados

Cada fold generará predicciones únicamente para su período fuera de muestra:

WF_01 → predicciones de 2022

WF_02 → predicciones de 2023

WF_03 → predicciones de 2024

Estas predicciones se unirán cronológicamente para obtener una serie fuera de muestra:

2022 + 2023 + 2024

Los modelos se compararán utilizando:

- resultado promedio entre folds;
- variabilidad entre folds;
- métricas por clase;
- estabilidad por año;
- calidad de las probabilidades.

No se seleccionará un modelo únicamente por su mejor resultado en un año.

6.8. Test final reservado

Los datos posteriores a 2024 permanecerán fuera de la comparación inicial.

Conceptualmente:

2020–2024

→ desarrollo y comparación walk-forward.

2025–2026

→ test final reservado.

El test final no se utilizará para:

- seleccionar arquitecturas;
- ajustar hiperparámetros;
- elegir la cantidad de épocas;
- seleccionar features;
- comparar ventanas temporales.

Se utilizará solamente después de seleccionar el modelo y la configuración final.

6.9. Configuración inicial

```
WALK_FORWARD_CONFIG = {
    "folds": {
        "WF_01": {
            "train_years": [2020, 2021],
            "validation_year": 2022,
            "internal_validation_year": 2021,
            "internal_validation_months": [10, 11, 12],
        },
        "WF_02": {
            "train_years": [2020, 2021, 2022],
            "validation_year": 2023,
            "internal_validation_year": 2022,
            "internal_validation_months": [10, 11, 12],
        },
        "WF_03": {
            "train_years": [2020, 2021, 2022, 2023],
            "validation_year": 2024,
            "internal_validation_year": 2023,
            "internal_validation_months": [10, 11, 12],
        },
    },
    "split_by_complete_trading_days": True,
    "shuffle": False,
    "use_internal_validation": True,
    "retrain_with_full_fold_train": True,
    "reserve_final_test_from_year": 2025,
}
```

6.10. Decisión metodológica

Cada fold seguirá este procedimiento:

1. Dividir cronológicamente el train general.
2. Entrenar con train interno.
3. Aplicar early stopping con validation interna.
4. Identificar la mejor cantidad de épocas.
5. Crear un modelo nuevo.
6. Entrenar con todo el train general.
7. Evaluar sobre la validación walk-forward.
8. Guardar métricas, probabilidades y predicciones.

De esta manera, el año de validación walk-forward representa una evaluación realmente fuera de muestra, mientras que la validación interna se utiliza exclusivamente para controlar el entrenamiento.

7. Modelos que se evaluarán

7.1. Objetivo

El Stage_07 evaluará modelos capaces de procesar ventanas temporales intradiarias para predecir las cinco clases del target OPC.

Todos los modelos utilizarán:

- El mismo target
- Las mismas features
- Las mismas ventanas
- Los mismos folds
- Las mismas métricas

La diferencia principal será la forma en que cada arquitectura procesa la secuencia histórica.

7.2. Modelos de referencia

Antes de comparar arquitecturas neuronales se conservarán dos referencias:

- Dummy Classifier
- Mejor modelo tabular del Stage_06

Estos modelos no constituyen el objetivo principal del stage, pero permiten comprobar si las arquitecturas secuenciales aportan una mejora real.

7.3. MLP

El **Multilayer Perceptron** será el baseline neuronal.

El dataset secuencial completo tendrá una estructura tridimensional:

```
X.shape = (
    n_samples,
    lookback,
    n_features
)
```

Por ejemplo:

```
X.shape = (500000, 60, 18)
```

Esto representa:

- 500000 ventanas
- 60 minutos por ventana
- 18 features por minuto

Cada ventana individual será una matriz bidimensional:

```
window.shape = (60, 18)
```

Sin embargo, el MLP no recibe directamente matrices temporales. Por ello, cada ventana se convertirá en un único vector:

```
60 × 18 = 1080 valores
```

La transformación será:

Ventana original:
(60, 18)

Ventana aplanada para el MLP:
(1080,)

Por lo tanto, el dataset completo de entrada al MLP tendrá una estructura bidimensional:

```
X_mlp.shape = (
    n_samples,
    lookback * n_features
)
```

Ejemplo:

```
X_mlp.shape = (500000, 1080)
```

El MLP podrá utilizar todos los valores contenidos en la ventana, pero no tendrá una arquitectura especializada para interpretar explícitamente la relación temporal entre minutos consecutivos.

Su función será responder:

¿Una red neuronal común puede aprovechar la información de la ventana
aunque esta se presente como un único vector?

7.4. CNN 1D

La **CNN 1D** aplicará filtros que recorren la secuencia temporal.

Buscará patrones locales como:

- Aumento rápido de volatilidad
- Secuencia de retornos positivos
- Compresión seguida de expansión
- Cambio reciente de momentum

La CNN está diseñada para detectar patrones cercanos dentro de una estructura temporal regular.

Su principal ventaja es que puede entrenarse de manera eficiente y con menos parámetros que una red completamente conectada equivalente.

7.5. LSTM

La **Long Short-Term Memory** procesará la ventana minuto a minuto y mantendrá un estado interno que resume la información anterior.

Su objetivo será aprender relaciones como:

Lo ocurrido al comienzo de la ventana
puede influir sobre la interpretación del estado final.

Las LSTM fueron diseñadas para capturar dependencias secuenciales y reducir los problemas de gradientes presentes en las redes recurrentes simples.

Será una de las arquitecturas principales del Stage_07.

7.6. GRU

La **Gated Recurrent Unit** tiene una lógica parecida a la LSTM, pero utiliza una estructura interna más simple.

Sus ventajas potenciales son:

- Menor cantidad de parámetros
- Entrenamiento más rápido
- Menor consumo de memoria
- Menor riesgo de sobreajuste

La comparación LSTM frente a GRU permitirá comprobar si el problema necesita una memoria recurrente más compleja o si una arquitectura más simple resulta suficiente.

7.7. TCN

La **Temporal Convolutional Network** utiliza convoluciones causales y dilatadas.

Las convoluciones causales aseguran que cada posición se construya solamente con información anterior dentro de la secuencia.

Las convoluciones dilatadas permiten observar partes más alejadas de la ventana sin necesitar una red excesivamente profunda.

Conceptualmente:

- Capas iniciales:
patrones de pocos minutos.

- Capas posteriores:
patrones de mayor alcance temporal.

La TCN será especialmente interesante porque combina:

- Procesamiento temporal
- Entrenamiento paralelo
- Memoria de mayor alcance

7.8. Catálogo inicial

El catálogo principal será:

```
MODEL_CATALOG = {  
    "mlp": {  
        "model_family": "feedforward",  
        "uses_temporal_structure": False,  
        "input_format": "flattened_window",
```

```

    },
    "cnn1d": {
        "model_family": "convolutional",
        "uses_temporal_structure": True,
        "input_format": "sequence",
    },
    "lstm": {
        "model_family": "recurrent",
        "uses_temporal_structure": True,
        "input_format": "sequence",
    },
    "gru": {
        "model_family": "recurrent",
        "uses_temporal_structure": True,
        "input_format": "sequence",
    },
    "tcn": {
        "model_family": "temporal_convolutional",
        "uses_temporal_structure": True,
        "input_format": "sequence",
    },
}

```

7.9. Modelos que no se evaluarán inicialmente

No incorporaría todavía:

Transformer
 CNN-LSTM
 BiLSTM
 Autoencoder
 GAN
 Reinforcement Learning

No porque sean modelos inválidos, sino porque aumentarían demasiado el número de experimentos antes de comprobar que las secuencias aportan señal estable.

Los autoencoders y GAN tienen objetivos diferentes, relacionados principalmente con representación o generación de datos.

El reinforcement learning tampoco es un clasificador OPC: aprende decisiones mediante recompensas e interacción con un entorno, por lo que correspondería a otro stage.

Un Transformer temporal podría evaluarse después si los modelos iniciales demuestran que existe señal secuencial suficiente.

7.10. Orden de evaluación

El orden recomendado será:

1. Dummy Classifier
2. Mejor modelo tabular del Stage_06
3. MLP

4. CNN 1D
5. GRU
6. LSTM
7. TCN

La GRU puede ejecutarse antes que la LSTM porque normalmente es más liviana y permite comprobar rápidamente si una arquitectura recurrente aporta valor.

7.11. Decisión metodológica

Los cinco modelos neuronales principales serán:

MLP
CNN 1D
GRU
LSTM
TCN

Pero la comparación final también deberá incluir:

Dummy Classifier
Mejor modelo tabular del Stage_06

Por lo tanto, tendremos **cinco arquitecturas nuevas** y **dos referencias**.

Solo se incorporarán modelos adicionales si los resultados iniciales muestran una mejora consistente y estable frente a los baselines.

8. Protocolo estándar de entrenamiento

8.1. Objetivo

Todos los modelos del Stage_07 deberán entrenarse utilizando el mismo protocolo experimental.

Esto permitirá que las diferencias observadas en los resultados provengan principalmente de la arquitectura del modelo y no de cambios en:

- los datos;
- las ventanas;
- los folds;
- el preprocesamiento;
- las métricas;
- las condiciones de entrenamiento.

La regla general será:

Mismos datos + mismas reglas + diferente arquitectura

8.2. Reglas comunes

Todos los modelos utilizarán:

- Mismo target OPC
- Mismas clases
- Mismas features
- Mismas ventanas temporales
- Mismos folds walk-forward
- Mismo preprocesamiento
- Mismas métricas
- Misma semilla inicial
- Mismo criterio de early stopping
- Mismo procedimiento de guardado

No se permitirá que un modelo utilice información adicional que no esté disponible para los demás.

8.3. Reproducibilidad

Se utilizará inicialmente una semilla fija:

```
RANDOM_SEED = 42
```

La semilla deberá aplicarse, cuando corresponda, a:

- Python
- NumPy
- Framework de deep learning
- Inicialización de pesos
- Orden de los batches

Esto permitirá reducir variaciones aleatorias entre ejecuciones.

Sin embargo, una misma semilla no garantiza resultados completamente idénticos cuando se utiliza GPU.

Por esta razón, después de identificar los mejores modelos se podrán repetir los experimentos con varias semillas para comprobar su estabilidad.

8.4. Entrenamiento por fold

Cada modelo se entrenará de forma independiente para cada fold:

- WF_01
- WF_02
- WF_03

El procedimiento será:

1. Ajustar el preprocesamiento con train interno.
2. Entrenar utilizando train interno.
3. Controlar el entrenamiento con validation interna.
4. Determinar la mejor época.
5. Crear un modelo nuevo.
6. Ajustar nuevamente el preprocesamiento con todo el train general.

7. Entrenar durante la cantidad de épocas seleccionada.
8. Evaluar sobre la validación walk-forward.

Los pesos aprendidos en un fold no se reutilizarán en el siguiente.

Cada fold comenzará con un modelo nuevo.

8.5. Early stopping

El entrenamiento tendrá una cantidad máxima de épocas:

`MAX_EPOCHS = 50`

Después de cada época se evaluará el desempeño sobre la validation interna.

Si la métrica seleccionada no mejora durante varias épocas consecutivas, el entrenamiento se detendrá.

La configuración inicial será:

`EARLY_STOPPING_PATIENCE = 5`

Ejemplo:

Época 10 → mejor resultado
Época 11 → no mejora
Época 12 → no mejora
Época 13 → no mejora
Época 14 → no mejora
Época 15 → no mejora

En este caso, el entrenamiento se detendrá después de cinco épocas sin mejora.

Se conservará la época que haya obtenido el mejor resultado, no necesariamente la última ejecutada.

8.6. Métrica utilizada durante el entrenamiento

La función de pérdida principal será la entropía cruzada multiclase:

`Multiclass Cross-Entropy Loss`

Esta función permite entrenar al modelo para producir probabilidades para las cinco clases OPC.

El early stopping no deberá controlarse únicamente con `accuracy`.

Inicialmente se utilizará una métrica que tenga en cuenta todas las clases, como:

`EARLY_STOPPING_METRIC = "validation_macro_f1"`

Si el framework utilizado no permite calcular directamente Macro F1 durante cada época, se podrá utilizar:

`EARLY_STOPPING_METRIC = "validation_loss"`

y calcular Macro F1 al finalizar cada época mediante una función adicional.

La decisión deberá mantenerse igual para todos los modelos.

8.7. Tamaño de batch

La configuración inicial será:

`BATCH_SIZE = 512`

Esto significa que el modelo procesará 512 ventanas antes de actualizar sus pesos.

Este valor busca un equilibrio entre:

- uso de memoria;
- velocidad de entrenamiento;
- estabilidad del gradiente.

El batch size podrá reducirse si alguna arquitectura supera la memoria disponible de la GPU.

En ese caso, el cambio deberá documentarse claramente.

No se aumentará o reducirá el batch size con el único objetivo de favorecer artificialmente a un modelo.

8.8. Desbalance de clases

Antes de entrenar cada fold se calculará la distribución de las clases utilizando únicamente el train correspondiente.

Si existe un desbalance importante, se podrán utilizar pesos de clase:

Clases frecuentes → menor peso

Clases poco frecuentes → mayor peso

Los pesos deberán calcularse exclusivamente con el train.

Conceptualmente:

```
class_weights = compute_class_weights(y_train)
```

Los datos de validation o test no deberán utilizarse para calcular estos pesos.

El mismo criterio deberá aplicarse a todos los modelos.

8.9. Optimización inicial

Todos los modelos comenzarán utilizando una configuración de optimización comparable.

Configuración inicial:

```
OPTIMIZER = "Adam"  
LEARNING_RATE = 1e-3  
LOSS_FUNCTION = "categorical_crossentropy"
```

La arquitectura podrá requerir pequeños ajustes, pero durante la primera comparación se evitará una optimización extensa de hiperparámetros.

El objetivo inicial será comparar modelos bajo configuraciones simples y controladas.

8.10. Guardado del mejor modelo

Durante el entrenamiento interno se guardará el modelo correspondiente a la mejor época.

Para cada fold se conservarán:

- Pesos del mejor modelo interno
- Mejor época
- Historial de entrenamiento
- Métrica de train
- Métrica de validation interna
- Configuración utilizada

Después del entrenamiento definitivo se guardará:

- Modelo final del fold
- Escalador o preprocesador
- Predicciones fuera de muestra
- Probabilidades por clase
- Métricas del fold

El nombre de los archivos deberá identificar:

- Modelo
- Dataset
- Lookback
- Fold
- Semilla

Ejemplo:

```
lstm__opc_reduced_no_level__lb60__WF_01__seed42
```

8.11. Información registrada

Cada entrenamiento deberá guardar como mínimo:

- Número de épocas ejecutadas
- Mejor época
- Loss de train
- Loss de validation interna
- Macro F1
- Balanced Accuracy
- Tiempo de entrenamiento
- Cantidad de parámetros
- Tamaño del batch
- Learning rate
- Semilla
- Fold

También se conservarán las curvas de entrenamiento para detectar:

- sobreajuste;
- entrenamiento insuficiente;
- inestabilidad;
- falta de convergencia.

8.12. Configuración inicial

```
TRAINING_CONFIG = {
    "random_seed": 42,
    "max_epochs": 50,
    "early_stopping_patience": 5,
    "early_stopping_metric": "validation_macro_f1",
    "restore_best_weights": True,
    "batch_size": 512,
    "optimizer": "Adam",
    "learning_rate": 1e-3,
    "loss_function": "categorical_crossentropy",
    "use_class_weights": True,
    "shuffle_train_batches": True,
    "shuffle_validation": False,
    "retrain_with_full_fold_train": True,
    "save_best_model": True,
    "save_training_history": True,
    "save_predictions": True,
    "save_probabilities": True,
}
```

El orden temporal de los folds no se modificará.

La opción:

"shuffle_train_batches": True

solo altera el orden de las ventanas dentro del train durante el entrenamiento. No mezcla observaciones entre train, validation y test.

8.13. Decisión metodológica

La primera comparación utilizará:

```
RANDOM_SEED = 42  
MAX_EPOCHS = 50  
EARLY_STOPPING_PATIENCE = 5  
BATCH_SIZE = 512  
LEARNING_RATE = 1e-3
```

Estos valores representan una configuración inicial común.

No se consideran valores definitivos.

La optimización detallada se realizará únicamente sobre los modelos que demuestren:

- Mejora frente a los baselines
- Estabilidad entre folds
- Buen desempeño en todas las clases
- Ausencia de sobreajuste evidente

La prioridad inicial será garantizar una comparación justa, reproducible y temporalmente correcta.

9. Métricas de evaluación

9.1. Objetivo

Todos los modelos se evaluarán utilizando las mismas métricas sobre las predicciones fuera de muestra de cada fold walk-forward.

La métrica principal será:

```
PRIMARY_METRIC = "macro_f1"
```

Sin embargo, ningún modelo será seleccionado utilizando únicamente un valor.

También se analizarán:

- desempeño general;
- desempeño de cada clase;
- estabilidad entre folds;
- distribución de las predicciones;
- calidad de las probabilidades;
- comportamiento por año y régimen.

9.2. Macro F1

Macro F1 será la métrica principal.

Primero se calcula el F1-score individual de cada clase:

LONG_TP
LONG_SL
SHORT_TP
SHORT_SL
NO_TRADE

Luego se obtiene el promedio simple:

$$\text{Macro F1} = \frac{(F1_LONG_TP + F1_LONG_SL + F1_SHORT_TP + F1_SHORT_SL + F1_NO_TRADE)}{5}$$

Todas las clases tienen la misma importancia, aunque algunas aparezcan con menor frecuencia.

Esto evita que una clase muy frecuente, como posiblemente **NO_TRADE**, domine completamente la evaluación.

Ejemplo:

F1 LONG_TP = 0.48
F1 LONG_SL = 0.30
F1 SHORT_TP = 0.46
F1 SHORT_SL = 0.28
F1 NO_TRADE = 0.65

$$\text{Macro F1} = 0.43$$

Un buen **Macro F1** requiere que el modelo funcione razonablemente bien en todas las clases.

9.3. Balanced Accuracy y Accuracy

Balanced Accuracy

La **Balanced Accuracy** calcula el promedio del recall obtenido en cada clase.

Permite responder:

¿El modelo identifica correctamente una proporción razonable de observaciones de cada clase?

Es útil cuando las clases no están balanceadas.

Accuracy

La **Accuracy** representa el porcentaje total de predicciones correctas:

$$\text{Predicciones correctas} / \text{Total de predicciones}$$

Se conservará como métrica descriptiva, pero no será la métrica principal.

Por ejemplo, si **NO_TRADE** representa el 60 % del dataset, un modelo que siempre prediga **NO_TRADE** podría obtener:

$$\text{Accuracy} = 0.60$$

aunque no sea capaz de identificar ninguna operación LONG o SHORT.

Por eso siempre se comparará con **Macro F1** , **Balanced Accuracy** y las métricas por clase.

9.4. Log Loss

La **Log Loss** evaluará la calidad de las probabilidades generadas por el modelo.

No solamente verifica si la clase final fue correcta, sino también qué nivel de confianza asignó el modelo.

Ejemplo:

Caso real: LONG_TP

Modelo A:

$P(\text{LONG_TP}) = 0.80$

Modelo B:

$P(\text{LONG_TP}) = 0.25$

Aunque ambos puedan terminar prediciendo correctamente en algunos casos, el modelo A asigna una probabilidad más coherente.

La interpretación será:

Menor Log Loss = mejores probabilidades

La Log Loss penaliza especialmente las predicciones incorrectas realizadas con demasiada confianza.

Esto será importante porque posteriormente las reglas operativas utilizarán probabilidades y umbrales de confianza.

9.5. Métricas por clase

Para cada clase se calcularán:

Precision
Recall
F1-score
Support

Precision

Indica cuántas predicciones de una clase fueron realmente correctas.

Ejemplo:

El modelo predijo LONG_TP 100 veces.
60 eran realmente LONG_TP.

$\text{Precision LONG_TP} = 0.60$

Permite responder:

Cuando el modelo predice esta clase, ¿cuántas veces acierta?

Recall

Indica cuántos casos reales de una clase fueron detectados.

Ejemplo:

Existían 100 casos reales de LONG_TP.
El modelo identificó 50.

$\text{Recall LONG_TP} = 0.50$

Permite responder:

De todos los casos reales de esta clase, ¿cuántos encontró?

F1-score

Combina Precision y Recall.

Un F1 alto requiere que ambas sean razonables.

Support

Representa la cantidad real de observaciones de cada clase.

No es una métrica de rendimiento, pero permite interpretar los resultados correctamente.

9.6. Diagnósticos obligatorios

Además de las métricas numéricas, se generarán los siguientes diagnósticos.

Matriz de confusión

Mostrará qué clases reales se confunden con qué clases predichas.

Será especialmente importante analizar errores como:

LONG_TP → predicho como LONG_SL
SHORT_TP → predicho como SHORT_SL
LONG → confundido con SHORT
Señal → confundida con NO_TRADE

No todos los errores tienen el mismo significado operativo.

Distribución de predicciones

Se comparará:

Distribución real de clases
vs.
Distribución predicha por el modelo

Esto permitirá detectar modelos que:

- predicen casi siempre `NO_TRADE` ;
- ignoran alguna clase;
- producen demasiadas señales LONG;
- producen demasiadas señales SHORT;
- colapsan hacia una única categoría.

Probabilidades predichas

Se analizarán:

- probabilidad máxima de cada predicción;
- distribución de confianza;
- confianza de predicciones correctas;
- confianza de predicciones incorrectas;
- probabilidades por clase.

Un modelo demasiado confiado en errores puede ser peligroso aunque tenga un Macro F1 aceptable.

9.7. Resultados por fold, año y régimen

Las métricas se calcularán por separado para:

WF_01 → 2022
WF_02 → 2023
WF_03 → 2024

Para cada modelo se reportará:

Macro F1 por fold
Balanced Accuracy por fold
Log Loss por fold
Accuracy por fold
Métricas por clase y fold

Luego se calcularán:

Promedio entre folds
Desviación estándar entre folds
Mejor fold
Peor fold

También se analizarán los resultados según el régimen intradiario:

Overnight
Pre-market
Opening
Regular
Closing

Esto permitirá comprobar si un modelo funciona de forma general o si sus resultados dependen excesivamente de un horario específico.

9.8. Criterio de selección

Un modelo no será seleccionado solamente por obtener el mayor **Macro F1** promedio.

También deberá cumplir:

- superar los baselines;
- mantener resultados razonables en los tres folds;
- no colapsar hacia **NO_TRADE** ;
- identificar tanto clases LONG como SHORT;
- mantener desempeño aceptable en clases TP y SL;
- producir probabilidades útiles;
- no presentar una variabilidad temporal excesiva.

Ejemplo:

Modelo A:

Macro F1 = 0.42, 0.41, 0.40

Modelo B:

Macro F1 = 0.34, 0.50, 0.25

Aunque el modelo B tenga el mejor fold individual, el modelo A sería más estable y potencialmente más confiable.

La selección considerará:

Rendimiento promedio + estabilidad + comportamiento por clase

9.9. Métricas operativas preliminares

Sin realizar todavía el backtesting definitivo, se podrán calcular indicadores preliminares como:

Porcentaje de predicciones **NO_TRADE**

Porcentaje de señales LONG

Porcentaje de señales SHORT

Precision de LONG_TP

Precision de SHORT_TP

Relación entre señales TP y SL predichas

Estas métricas permitirán interpretar el comportamiento operativo del modelo.

No se calcularán todavía como resultado final:

- beneficio acumulado;
- Sharpe Ratio;
- drawdown;
- comisiones;
- slippage;
- tamaño de posición.

Estas medidas corresponden a la etapa posterior de backtesting.

9.10. Configuración inicial

```
METRICS_CONFIG = {
    "primary_metric": "macro_f1",
    "general_metrics": [
        "macro_f1",
        "balanced_accuracy",
        "log_loss",
        "accuracy",
    ],
    "per_class_metrics": [
        "precision",
        "recall",
        "f1_score",
        "support",
    ],
    "diagnostics": [
        "confusion_matrix",
        "true_class_distribution",
        "predicted_class_distribution",
        "predicted_probabilities",
        "prediction_confidence",
    ],
    "grouped_evaluations": [
        "fold",
        "year",
        "regime_id",
    ],
    "aggregate_across_folds": [
        "mean",
        "standard_deviation",
        "minimum",
        "maximum",
    ],
}
```

9.11. Decisión metodológica

La métrica principal será:

Macro F1

pero la selección final considerará conjuntamente:

Macro F1
 Balanced Accuracy
 Log Loss
 Métricas por clase
 Matriz de confusión
 Estabilidad entre folds
 Distribución de predicciones
 Calidad de las probabilidades

La **Accuracy** se utilizará únicamente como información complementaria.

Un modelo será considerado útil solamente si supera los baselines y mantiene un comportamiento equilibrado y estable fuera de muestra.

10. Criterios de comparación

10.1. Objetivo

Los modelos se compararán utilizando sus resultados fuera de muestra en los tres folds walk-forward:

WF_01 → 2022

WF_02 → 2023

WF_03 → 2024

Un modelo no será considerado mejor únicamente por obtener el mayor resultado en un fold.

Buscaremos un modelo que funcione razonablemente bien en diferentes períodos de mercado.

10.2. Criterios principales

La comparación considerará:

Macro F1 promedio
Variabilidad entre folds
Peor resultado entre folds
Métricas por clase
Distribución de predicciones
Calidad de las probabilidades
Estabilidad por año y régimen

La prioridad será:

Buen rendimiento promedio
+
Resultados estables
+
Comportamiento equilibrado entre clases

10.3. Rendimiento y estabilidad

Para cada modelo se calculará:

Macro F1 promedio
Desviación estándar
Macro F1 mínimo
Macro F1 máximo

Ejemplo:

Modelo A:

Macro F1 = 0.42, 0.41, 0.40

Promedio = 0.41

Variabilidad baja

Peor fold = 0.40

Modelo B:

Macro F1 = 0.37, 0.49, 0.25

Promedio = 0.37

Variabilidad alta

Peor fold = 0.25

Aunque el modelo B obtiene el mejor resultado individual:

0.49

el modelo A sería preferible porque mantiene un desempeño similar en todos los años.

La idea será:

No buscamos el mejor año.

Buscamos el modelo más confiable a través del tiempo.

10.4. Condiciones de rechazo

Un modelo podrá ser descartado aunque tenga un Macro F1 promedio aceptable si presenta alguno de estos problemas:

No supera los baselines

Colapsa hacia NO_TRADE

No predice una o más clases

Funciona bien solamente en un fold

Presenta resultados muy inestables

Tiene Log Loss excesivamente alta

Produce probabilidades demasiado confiadas y erróneas

También se revisará que el modelo no obtenga un buen resultado únicamente porque funciona bien en la clase más frecuente.

10.5. Orden de comparación

La selección se realizará en este orden:

1. Verificar que supere al Dummy Classifier.
2. Comparar contra el mejor modelo tabular.
3. Ordenar por Macro F1 promedio.
4. Revisar la variabilidad entre folds.
5. Revisar el peor fold.
6. Analizar las métricas por clase.
7. Analizar Log Loss y probabilidades.
8. Revisar estabilidad por régimen.

No utilizaremos inicialmente una fórmula única que combine todas las métricas, porque podría ocultar problemas importantes.

Cada criterio se analizará por separado.

10.6. Comparación por clase

Dos modelos con el mismo Macro F1 promedio pueden comportarse de manera muy diferente.

Ejemplo:

Modelo A:

Buen desempeño en LONG_TP, SHORT_TP y NO_TRADE.

Muy bajo desempeño en LONG_SL y SHORT_SL.

Modelo B:

Desempeño moderado pero equilibrado en las cinco clases.

En este caso, el modelo B podría ser más útil aunque su Macro F1 sea ligeramente inferior.

Se prestará especial atención a:

LONG_TP frente a LONG_SL

SHORT_TP frente a SHORT_SL

Señales operativas frente a NO_TRADE

Esto permitirá detectar si el modelo reconoce correctamente la dirección, pero falla al diferenciar entre TP y SL.

10.7. Criterios de desempate

Cuando dos modelos tengan resultados similares, se priorizará:

1. Menor variabilidad entre folds.
2. Mejor resultado en el peor fold.
3. Mejor desempeño en las clases minoritarias.
4. Menor Log Loss.
5. Menor complejidad.
6. Menor tiempo de entrenamiento.

Por ejemplo, si una GRU y una LSTM obtienen resultados prácticamente iguales, se podrá preferir la GRU si:

Tiene menos parámetros

Entrena más rápido

Consume menos memoria

Mantiene la misma estabilidad

No se elegirá un modelo más complejo si no produce una mejora clara.

10.8. Tabla comparativo

La comparación final tendrá una estructura similar a:

Modelo	Macro F1 medio	Desv. estándar	Peor fold	Balanced Accuracy	Log Loss	Colapso
Dummy	0.18	0.01	0.17	0.20	1.45	Sí
Tabular	0.35	0.03	0.31	0.39	1.21	No
MLP	0.38	0.04	0.33	0.41	1.15	No
CNN 1D	0.41	0.02	0.39	0.44	1.08	No
LSTM	0.40	0.07	0.31	0.43	1.12	No

Los valores anteriores son solamente ilustrativos.

La tabla real se construirá con los resultados obtenidos durante los experimentos.

10.9. Configuración inicial

```
COMPARISON_CONFIG = {
    "primary_metric": "macro_f1",
    "compare_mean_across_folds": True,
    "compare_std_across_folds": True,
    "compare_worst_fold": True,
    "require_baseline_improvement": True,
    "check_per_class_metrics": True,
    "check_prediction_distribution": True,
    "check_probability_quality": True,
    "check_regime_stability": True,
    "prefer_simpler_model_on_tie": True,
}
```

10.10. Decisión metodológica

El mejor modelo será aquel que combine:

- Buen Macro F1 promedio
- Baja variabilidad entre folds
- Buen resultado en el peor año
- Resultados razonables en todas las clases
- Ausencia de colapso hacia NO_TRADE
- Probabilidades útiles
- Estabilidad temporal

La decisión final no se basará únicamente en:

- El mayor valor individual

sino en:

Rendimiento + estabilidad + equilibrio + simplicidad

11. Estructura estándar de las notebooks de modelos

11.1. Objetivo

Cada arquitectura tendrá su propia notebook, pero todas seguirán exactamente la misma estructura.

Esto permitirá comparar los modelos bajo un procedimiento común y evitar que cada notebook termine usando:

- Datos diferentes
- Preprocesamientos diferentes
- Folds diferentes
- Métricas diferentes
- Criterios de guardado diferentes

La única diferencia importante entre notebooks deberá ser la arquitectura del modelo.

11.2. Estructura común

Cada notebook tendrá las siguientes secciones:

1. Objetivo
2. Configuración
3. Carga de datos
4. Validación del dataset
5. Construcción del modelo
6. Entrenamiento por fold
7. Evaluación por fold
8. Agregación de resultados
9. Diagnósticos
10. Guardado
11. Observaciones

11.3. Contenido de cada sección

1. Objetivo

Se indicará:

- Modelo evaluado
- Target utilizado
- Dataset utilizado
- Lookback utilizado
- Pregunta que busca responder el experimento

Ejemplo:

- Evaluar si una CNN 1D puede aprovechar patrones locales dentro de ventanas de 60 minutos para predecir el target OPC.

2. Configuración

Se cargarán todas las configuraciones comunes:

- Semilla
- Folds
- Lookback
- Features
- Batch size
- Máximo de épocas
- Early stopping
- Learning rate
- Métricas
- Rutas de entrada y salida

Las configuraciones compartidas no se redefinirán arbitrariamente dentro de cada notebook.

3. Carga de datos

Se cargarán:

- Dataset secuencial
- Target
- Metadatos de las ventanas
- Definición de folds
- Lista de features
- Codificación de clases

También se imprimirá un resumen básico:

- Shape de X
- Shape de y
- Rango temporal
- Cantidad de features
- Distribución de clases

4. Validación del dataset

Antes de entrenar se verificará:

- Alineación entre X, y y metadatos
- Ausencia de NaN e infinitos
- Dimensiones correctas
- Orden temporal
- Clases válidas
- Features esperadas
- Separación correcta de los folds

Si alguna validación falla, el entrenamiento no deberá continuar.

5. Construcción del modelo

Se definirá la arquitectura específica de la notebook.

Se mostrará:

- Forma de entrada
- Capas

- Cantidad de unidades o filtros
- Activaciones
- Regularización
- Capa de salida
- Cantidad de parámetros

La salida siempre tendrá cinco probabilidades:

- Una probabilidad por cada clase OPC

6. Entrenamiento por fold

Para cada fold se ejecutará el protocolo estándar:

- Train interno
- Validation interna
- Early stopping
- Selección de la mejor época
- Reentrenamiento con todo el train del fold

Cada fold utilizará un modelo nuevo inicializado desde cero.

7. Evaluación por fold

Sobre la validación walk-forward se calcularán:

- Macro F1
- Balanced Accuracy
- Log Loss
- Accuracy
- Métricas por clase
- Matriz de confusión

También se guardarán:

- Clase real
- Clase predicha
- Probabilidades
- Timestamp
- Régimen
- Fold

8. Agregación de resultados

Se unirán los resultados de:

- WF_01
- WF_02
- WF_03

Luego se calcularán:

- Promedio entre folds
- Desviación estándar
- Mejor fold
- Peor fold
- Resultados agregados 2022-2024

9. Diagnósticos

Se analizarán:

- Curvas de entrenamiento
- Sobreaajuste
- Matriz de confusión
- Distribución de predicciones
- Confianza de las probabilidades
- Resultados por clase
- Resultados por régimen
- Resultados por año

El objetivo será entender no solo cuánto acierta el modelo, sino también cómo se equivoca.

10. Guardado

Cada notebook deberá guardar:

- Configuración utilizada
- Modelo final de cada fold
- Preprocesadores
- Historial de entrenamiento
- Métricas
- Predicciones
- Probabilidades
- Matrices de confusión
- Resumen agregado

Los nombres deberán identificar:

- Modelo
- Dataset
- Lookback
- Fold
- Semilla

11. Observaciones

Al final se documentará:

- Qué funcionó
- Qué no funcionó
- Problemas encontrados
- Señales de sobreaajuste
- Clases más difíciles
- Estabilidad entre folds
- Recomendación para el siguiente experimento

Esta sección deberá basarse en los resultados obtenidos y no en impresiones previas.

11.4. Notebooks previstas

La estructura inicial será:

`S07_01_sequence_dataset.ipynb`

`S07_02_dummy_baseline.ipynb`

`S07_03_tabular_baseline.ipynb`

`S07_04_mlp.ipynb`

`S07_05_cnn1d.ipynb`

`S07_06_gru.ipynb`

`S07_07_lstm.ipynb`

`S07_08_tcn.ipynb`

`S07_09_model_comparison.ipynb`

La notebook de comparación no entrenará modelos nuevos.

Su función será cargar y comparar los resultados guardados por las notebooks anteriores.

11.5. Regla metodológica

Todas las notebooks deberán reutilizar funciones comunes para:

Carga de datos

Creación de folds

Preprocesamiento

Cálculo de métricas

Guardado de resultados

Gráficos

No conviene copiar y modificar manualmente las mismas funciones en cada notebook, porque podrían aparecer diferencias accidentales.

La lógica común deberá ubicarse en módulos o funciones reutilizables.

11.6. Decisión metodológica

La estructura estándar será obligatoria para todos los modelos.

De esta manera:

El procedimiento será el mismo

Los resultados serán comparables

Los errores serán más fáciles de detectar

Las notebooks serán más simples de revisar

La arquitectura cambiará entre experimentos, pero el protocolo de evaluación permanecerá constante.

12. Archivos de salida

12.1. Objetivo

La notebook:

`S07_00_experimental_design.ipynb`

no entrenará modelos ni construirá ventanas temporales.

Su función será:

- Crear la estructura de carpetas
- Guardar la configuración experimental
- Guardar la definición de folds
- Guardar el catálogo de modelos
- Guardar la lista de métricas

Estos archivos serán utilizados por las notebooks siguientes para evitar redefinir manualmente las mismas reglas.

12.2. Estructura de carpetas

La carpeta principal del Stage_07 será:

```
07_mnq_models/  
├── config/  
│   ├── stage_07_experimental_config.json  
│   ├── stage_07_folds.csv  
│   ├── stage_07_models.csv  
│   └── stage_07_metrics.csv  
├── sequences/  
├── models/  
├── predictions/  
├── metrics/  
└── reports/
```

La notebook `S07_00` deberá crear todas estas carpetas, aunque inicialmente algunas permanezcan vacías.

12.3. Función de cada carpeta

`config/`

Contendrá las reglas comunes del Stage_07:

- Target
- Dataset
- Features
- Ventanas
- Folds
- Modelos
- Entrenamiento

Métricas

Rutas

Semillas

sequences/

Contendrá los datasets de ventanas temporales generados en:

S07_01_sequence_dataset.ipynb

Ejemplos:

X

y

Metadatos de cada ventana

Lista y orden de features

Resumen de validación

models/

Contendrá los modelos entrenados y sus preprocesadores:

Pesos del modelo

Configuración del modelo

Escalador

Codificación del target

Historial de entrenamiento

Los resultados deberán estar separados por modelo y fold.

predictions/

Contendrá las predicciones fuera de muestra:

Timestamp

Clase real

Clase predicha

Probabilidades por clase

Fold

Año

Régimen

metrics/

Contendrá:

Métricas por fold

Métricas por clase

Matrices de confusión

Resultados agregados

Comparación entre modelos

reports/

Contendrá informes y diagnósticos:

- Curvas de entrenamiento
- Gráficos
- Tablas comparativas
- Observaciones
- Resumen final del Stage_07

12.4. Archivo principal de configuración

El archivo principal será:

stage_07_experimental_config.json

Este archivo concentrará las decisiones metodológicas comunes.

Su estructura conceptual será:

```
{
  "stage": "stage_07",
  "target": {
    "name": "opc_p50_h60_tp15_sl10",
    "problem_type": "multiclass_classification",
    "n_classes": 5
  },
  "dataset": {
    "feature_set": "OPC_reduced_no_level",
    "scope": "all_regimes",
    "feature_source": "stage_05_stage_06_metadata"
  },
  "windows": {
    "candidate_lookbacks": [30, 60, 90],
    "primary_lookback": 60,
    "allow_cross_day": false,
    "allow_time_gaps": false,
    "allow_cross_contract": false,
    "allow_cross_regime": true
  },
  "walk_forward": {
    "folds_file": "stage_07_folds.csv",
    "use_internal_validation": true,
    "reserve_final_test_from_year": 2025
  },
  "training": {
    "random_seed": 42,
    "max_epochs": 50,
    "early_stopping_patience": 5,
    "batch_size": 512,
    "optimizer": "Adam",
    "learning_rate": 0.001
  },
  "evaluation": {
```

```
    "primary_metric": "macro_f1",
    "metrics_file": "stage_07_metrics.csv"
  },
  "models": {
    "catalog_file": "stage_07_models.csv"
  }
}
```

La estructura exacta podrá ajustarse al momento de implementarla, pero deberá contener todas las decisiones necesarias para reproducir el experimento.

12.5. Archivo de folds

El archivo:

`stage_07_folds.csv`

tendrá una fila por fold.

Columnas mínimas:

```
fold_id
train_start_year
train_end_year
internal_validation_year
internal_validation_start_month
internal_validation_end_month
validation_year
```

Ejemplo:

fold_id	train_start_year	train_end_year	internal_validation_year	validation_year
WF_01	2020	2021	2021	2022
WF_02	2020	2022	2022	2023
WF_03	2020	2023	2023	2024

Las fechas exactas de trading se resolverán posteriormente utilizando el índice real del dataset.

12.6. Archivo de modelos

El archivo:

`stage_07_models.csv`

definirá el catálogo inicial.

Columnas mínimas:

```
model_id
model_name
model_family
input_format
uses_temporal_structure
```

enabled
notebook

Ejemplo:

model_id	model_name	input_format	enabled
dummy	Dummy Classifier	tabular	True
tabular	Tabular Baseline	tabular	True
mlp	MLP	flattened_window	True
cnn1d	CNN 1D	sequence	True
gru	GRU	sequence	True
lstm	LSTM	sequence	True
tcn	TCN	sequence	True

12.7. Archivo de métricas

El archivo:

`stage_07_metrics.csv`

indicará qué métricas deben calcular todas las notebooks.

Columnas mínimas:

metric_name
metric_level
primary_metric
optimization_direction
enabled

Ejemplo:

metric_name	metric_level	primary_metric	optimization_direction
macro_f1	general	True	maximize
balanced_accuracy	general	False	maximize
log_loss	general	False	minimize
accuracy	general	False	maximize
precision	class	False	maximize
recall	class	False	maximize
f1_score	class	False	maximize
support	class	False	none

12.8. Reglas de guardado

Todas las notebooks posteriores deberán:


```
Leer la configuración común desde config/  
No redefinir manualmente los folds  
No cambiar silenciosamente el target  
No cambiar silenciosamente las métricas  
Guardar los resultados en la carpeta correspondiente  
Registrar cualquier cambio experimental
```

Cada archivo de resultados deberá identificar como mínimo:

```
Modelo  
Dataset  
Lookback  
Fold  
Semilla  
Fecha de ejecución
```

Ejemplo:

```
gru__opc_reduced_no_level__lb60__WF_01__seed42
```

12.9. Resultado esperado de S07_00

Al finalizar la notebook se deberá imprimir un resumen similar a:

```
Stage_07 configurado correctamente
```

```
Carpetas creadas:
```

- config
- sequences
- models
- predictions
- metrics
- reports

```
Archivos guardados:
```

- stage_07_experimental_config.json
- stage_07_folds.csv
- stage_07_models.csv
- stage_07_metrics.csv

```
Estado final:
```

```
CONFIGURACIÓN APROBADA
```

12.10. Decisión metodológica

S07_00 será la fuente central de configuración del Stage_07.

Las notebooks posteriores deberán leer estos archivos para garantizar que todos los experimentos utilicen:

```
El mismo target  
Los mismos folds  
Las mismas ventanas  
Las mismas métricas  
Las mismas reglas generales
```

De esta manera evitaremos configuraciones duplicadas, cambios accidentales y resultados difíciles de comparar.

Códigos de implementación

Celda 12.1 — Rutas y carpetas

```
In [1]: # 12.1. Rutas y estructura de carpetas del Stage_07
# =====

from pathlib import Path
from datetime import datetime
from zoneinfo import ZoneInfo

import json
import pandas as pd

# -----
# Ruta principal del proyecto
# -----
# Si PROJECT_ROOT ya fue definido anteriormente, se reutiliza.
# En caso contrario, se utiliza esta ruta por defecto.

# Buscar la raíz del proyecto
PROJECT_ROOT = Path.cwd()

while PROJECT_ROOT.name != "neural_profit_local":
    PROJECT_ROOT = PROJECT_ROOT.parent

print("Project root:", PROJECT_ROOT)

if not PROJECT_ROOT.exists():
    raise FileNotFoundError(
        "No se encontró PROJECT_ROOT.\n"
        f"Ruta evaluada: {PROJECT_ROOT}\n"
        "Corrija la variable PROJECT_ROOT antes de continuar."
    )

# -----
# Carpeta general de datos
# -----
DATA_ROOT = Path(
    globals().get(
        "DATA_ROOT",
        PROJECT_ROOT / "data"
    )
)

# -----
# Carpeta principal del Stage_07
# -----
STAGE_07_ROOT = DATA_ROOT / "07_mnq_models"

STAGE_07_DIRS = {
```

```
"config": STAGE_07_ROOT / "config",
"sequences": STAGE_07_ROOT / "sequences",
"models": STAGE_07_ROOT / "models",
"predictions": STAGE_07_ROOT / "predictions",
"metrics": STAGE_07_ROOT / "metrics",
"reports": STAGE_07_ROOT / "reports",
}

# -----
# Crear carpetas
# -----
STAGE_07_ROOT.mkdir(parents=True, exist_ok=True)

for directory in STAGE_07_DIRS.values():
    directory.mkdir(parents=True, exist_ok=True)

# -----
# Mostrar resultado
# -----
print("=" * 80)
print("ESTRUCTURA DE CARPETAS DEL STAGE_07")
print("=" * 80)

print(f"\nPROJECT_ROOT:")
print(PROJECT_ROOT)

print(f"\nSTAGE_07_ROOT:")
print(STAGE_07_ROOT)

print("\nCarpetas disponibles:")

for name, directory in STAGE_07_DIRS.items():
    print(f"- {name:<12}: {directory}")

print("\nEstado: CARPETAS CREADAS CORRECTAMENTE")
```

Project root: c:\Users\heguu\OneDrive\Escritorio\neural_profit_local

=====

ESTRUCTURA DE CARPETAS DEL STAGE_07

=====

PROJECT_ROOT:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local

STAGE_07_ROOT:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models

Carpetas disponibles:

- config : c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\config
- sequences : c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\sequences
- models : c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\models
- predictions : c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\predictions
- metrics : c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\metrics
- reports : c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\reports

Estado: CARPETAS CREADAS CORRECTAMENTE

Celda 12.2 — Configuración y tablas auxiliares

La codificación exacta del target queda marcada como pendiente de validación contra los metadatos del Stage_04. No conviene asumirla silenciosamente.

```
In [2]: # 12.2. Definición de la configuración experimental
# =====

# -----
# Identificación del experimento
# -----

STAGE_NAME = "stage_07"

TARGET_NAME = "opc_p50_h60_tp15_sl10"

FEATURE_SET = "OPC_reduced_no_level"

DATASET_NAME = (
    f"{TARGET_NAME}"
    f"__{FEATURE_SET}"
    f"__all_regimes"
)

PRIMARY_LOOKBACK = 60
LOOKBACK_WINDOWS = [30, 60, 90]

PRIMARY_METRIC = "macro_f1"

CREATED_AT = datetime.now(
    ZoneInfo("America/Argentina/Buenos_Aires")
).isoformat(timespec="seconds")
```

```

# -----
# Definición de folds
# -----
stage_07_folds = pd.DataFrame(
    [
        {
            "fold_id": "WF_01",
            "train_start_year": 2020,
            "train_end_year": 2021,
            "internal_validation_year": 2021,
            "internal_validation_start_month": 10,
            "internal_validation_end_month": 12,
            "validation_year": 2022,
        },
        {
            "fold_id": "WF_02",
            "train_start_year": 2020,
            "train_end_year": 2022,
            "internal_validation_year": 2022,
            "internal_validation_start_month": 10,
            "internal_validation_end_month": 12,
            "validation_year": 2023,
        },
        {
            "fold_id": "WF_03",
            "train_start_year": 2020,
            "train_end_year": 2023,
            "internal_validation_year": 2023,
            "internal_validation_start_month": 10,
            "internal_validation_end_month": 12,
            "validation_year": 2024,
        },
    ]
)

# -----
# Catálogo inicial de modelos
# -----
stage_07_models = pd.DataFrame(
    [
        {
            "model_id": "dummy",
            "model_name": "Dummy Classifier",
            "model_family": "baseline",
            "input_format": "target_distribution",
            "uses_temporal_structure": False,
            "enabled": True,
            "notebook": "S07_02_dummy_baseline.ipynb",
        },
        {
            "model_id": "tabular",
            "model_name": "Tabular Baseline",
            "model_family": "tabular",
            "input_format": "tabular",
            "uses_temporal_structure": False,
            "enabled": True,
            "notebook": "S07_03_tabular_baseline.ipynb",
        },
    ]
)

```

```

    },
    {
        "model_id": "mlp",
        "model_name": "MLP",
        "model_family": "feedforward",
        "input_format": "flattened_window",
        "uses_temporal_structure": False,
        "enabled": True,
        "notebook": "S07_04_mlp.ipynb",
    },
    {
        "model_id": "cnn1d",
        "model_name": "CNN 1D",
        "model_family": "convolutional",
        "input_format": "sequence",
        "uses_temporal_structure": True,
        "enabled": True,
        "notebook": "S07_05_cnn1d.ipynb",
    },
    {
        "model_id": "gru",
        "model_name": "GRU",
        "model_family": "recurrent",
        "input_format": "sequence",
        "uses_temporal_structure": True,
        "enabled": True,
        "notebook": "S07_06_gru.ipynb",
    },
    {
        "model_id": "lstm",
        "model_name": "LSTM",
        "model_family": "recurrent",
        "input_format": "sequence",
        "uses_temporal_structure": True,
        "enabled": True,
        "notebook": "S07_07_lstm.ipynb",
    },
    {
        "model_id": "tcn",
        "model_name": "TCN",
        "model_family": "temporal_convolutional",
        "input_format": "sequence",
        "uses_temporal_structure": True,
        "enabled": True,
        "notebook": "S07_08_tcn.ipynb",
    },
]
)

# -----
# Catálogo de métricas
# -----
stage_07_metrics = pd.DataFrame(
    [
        {
            "metric_name": "macro_f1",
            "metric_level": "general",
            "primary_metric": True,
            "optimization_direction": "maximize",

```

```

        "enabled": True,
    },
    {
        "metric_name": "balanced_accuracy",
        "metric_level": "general",
        "primary_metric": False,
        "optimization_direction": "maximize",
        "enabled": True,
    },
    {
        "metric_name": "log_loss",
        "metric_level": "general",
        "primary_metric": False,
        "optimization_direction": "minimize",
        "enabled": True,
    },
    {
        "metric_name": "accuracy",
        "metric_level": "general",
        "primary_metric": False,
        "optimization_direction": "maximize",
        "enabled": True,
    },
    {
        "metric_name": "precision",
        "metric_level": "class",
        "primary_metric": False,
        "optimization_direction": "maximize",
        "enabled": True,
    },
    {
        "metric_name": "recall",
        "metric_level": "class",
        "primary_metric": False,
        "optimization_direction": "maximize",
        "enabled": True,
    },
    {
        "metric_name": "f1_score",
        "metric_level": "class",
        "primary_metric": False,
        "optimization_direction": "maximize",
        "enabled": True,
    },
    {
        "metric_name": "support",
        "metric_level": "class",
        "primary_metric": False,
        "optimization_direction": "none",
        "enabled": True,
    },
    ],
)

```

```

# -----
# Configuración general del Stage_07
# -----
stage_07_experimental_config = {
    "schema_version": "1.0",

```

```

"stage": STAGE_NAME,
"created_at": CREATED_AT,

"experiment": {
  "name": "mnq_opc_sequence_classification",
  "description": (
    "Comparación walk-forward de modelos tabulares y "
    "neuronales para el target OPC."
  ),
},

"target": {
  "name": TARGET_NAME,
  "problem_type": "multiclass_classification",
  "n_classes": 5,
  "encoding": "integer",
  "expected_class_mapping": {
    "0": "LONG_TP",
    "1": "LONG_SL",
    "2": "SHORT_TP",
    "3": "SHORT_SL",
    "4": "NO_TRADE",
  },
  "class_mapping_verified": False,
  "class_mapping_source": "stage_04_metadata",
},

"dataset": {
  "dataset_name": DATASET_NAME,
  "feature_set": FEATURE_SET,
  "scope": "all_regimes",
  "include_price_level_features": False,
  "feature_source": "stage_05_stage_06_metadata",
  "feature_columns": None,
  "predictive_context_columns": [
    "minute_of_day",
    "regime_id",
  ],
  "analytical_context_columns": [
    "trading_date",
    "year",
    "quarter",
    "contract",
  ],
},

"windows": {
  "candidate_lookbacks": LOOKBACK_WINDOWS,
  "primary_lookback": PRIMARY_LOOKBACK,
  "frequency_minutes": 1,
  "include_current_timestamp": True,
  "target_aligned_to_window_end": True,
  "allow_cross_day": False,
  "allow_time_gaps": False,
  "allow_cross_contract": False,
  "allow_cross_regime": True,
  "input_dtype": "float32",
},

"walk_forward": {

```



```

    "folds_file": "stage_07_folds.csv",
    "split_by_complete_trading_days": True,
    "shuffle_temporal_splits": False,
    "use_internal_validation": True,
    "internal_validation_method": "last_quarter_of_fold_train",
    "retrain_with_full_fold_train": True,
    "final_test_start_year": 2025,
    "final_test_end_year": 2026,
    "final_test_reserved": True,
},

"training": {
    "random_seed": 42,
    "max_epochs": 50,
    "early_stopping_patience": 5,
    "early_stopping_metric": "val_macro_f1",
    "early_stopping_mode": "max",
    "restore_best_weights": True,
    "batch_size": 512,
    "optimizer": "Adam",
    "learning_rate": 0.001,
    "loss_function": "sparse_categorical_crossentropy",
    "use_class_weights": True,
    "class_weights_source": "fold_train_only",
    "shuffle_train_batches": True,
    "shuffle_validation": False,
},

"evaluation": {
    "primary_metric": PRIMARY_METRIC,
    "metrics_file": "stage_07_metrics.csv",
    "grouped_evaluations": [
        "fold",
        "year",
        "regime_id",
    ],
    "save_predictions": True,
    "save_probabilities": True,
    "save_confusion_matrices": True,
},

"models": {
    "catalog_file": "stage_07_models.csv",
    "enabled_model_ids": (
        stage_07_models.loc[
            stage_07_models["enabled"],
            "model_id"
        ]
        .tolist()
    ),
},

"paths": {
    "stage_root": "data/07_mnq_models",
    "config": "data/07_mnq_models/config",
    "sequences": "data/07_mnq_models/sequences",
    "models": "data/07_mnq_models/models",
    "predictions": "data/07_mnq_models/predictions",
    "metrics": "data/07_mnq_models/metrics",
    "reports": "data/07_mnq_models/reports",
}

```

```

    },
    "validation_status": {
        "configuration_created": True,
        "target_mapping_verified": False,
        "feature_columns_verified": False,
        "sequence_dataset_created": False,
    },
}

print("=" * 80)
print("CONFIGURACIÓN EXPERIMENTAL CONSTRUIDA")
print("=" * 80)

print(f"\nDataset:")
print(DATASET_NAME)

print(f"\nTarget:")
print(TARGET_NAME)

print(f"\nLookback principal:")
print(PRIMARY_LOOKBACK)

print(f"\nModelos habilitados:")
print(stage_07_experimental_config["models"]["enabled_model_ids"])

print(f"\nMétrica principal:")
print(PRIMARY_METRIC)

```

```

=====
CONFIGURACIÓN EXPERIMENTAL CONSTRUIDA
=====

```

Dataset:

opc_p50_h60_tp15_sl10__OPC_reduced_no_level__all_regimes

Target:

opc_p50_h60_tp15_sl10

Lookback principal:

60

Modelos habilitados:

['dummy', 'tabular', 'mlp', 'cnn1d', 'gru', 'lstm', 'tcn']

Métrica principal:

macro_f1

Celda 12.3 — Validaciones antes del guardado

Estas validaciones evitan guardar una configuración internamente incoherente.

```

In [3]: # 12.3. Validación de la configuración
# =====

# -----
# Validaciones de ventanas
# -----

```

```
assert PRIMARY_LOOKBACK in LOOKBACK_WINDOWS, (  
    "PRIMARY_LOOKBACK no está dentro de LOOKBACK_WINDOWS."  
)  
  
assert len(LOOKBACK_WINDOWS) == len(set(LOOKBACK_WINDOWS)), (  
    "Existen lookbacks duplicados."  
)  
  
assert all(window > 0 for window in LOOKBACK_WINDOWS), (  
    "Todos los lookbacks deben ser positivos."  
)  
  
# -----  
# Validaciones de folds  
# -----  
assert stage_07_folds["fold_id"].is_unique, (  
    "Existen fold_id duplicados."  
)  
  
assert (  
    stage_07_folds["validation_year"]  
    > stage_07_folds["train_end_year"]  
) .all(), (  
    "El año de validación debe ser posterior al train."  
)  
  
assert (  
    stage_07_folds["internal_validation_year"]  
    == stage_07_folds["train_end_year"]  
) .all(), (  
    "La validación interna debe pertenecer al último año del train."  
)  
  
assert (  
    stage_07_folds["internal_validation_start_month"]  
    <= stage_07_folds["internal_validation_end_month"]  
) .all(), (  
    "Las fechas de validación interna son inconsistentes."  
)  
  
# -----  
# Validaciones de modelos  
# -----  
assert stage_07_models["model_id"].is_unique, (  
    "Existen model_id duplicados."  
)  
  
assert stage_07_models["notebook"].is_unique, (  
    "Una misma notebook fue asignada a más de un modelo."  
)  
  
assert stage_07_models["enabled"].any(), (  
    "No existe ningún modelo habilitado."  
)  
  
# -----  
# Validaciones de métricas
```

```

# -----
assert stage_07_metrics["metric_name"].is_unique, (
    "Existen métricas duplicadas."
)

assert stage_07_metrics["primary_metric"].sum() == 1, (
    "Debe existir exactamente una métrica principal."
)

saved_primary_metric = stage_07_metrics.loc[
    stage_07_metrics["primary_metric"],
    "metric_name"
].iloc[0]

assert saved_primary_metric == PRIMARY_METRIC, (
    "La métrica principal del CSV no coincide con la configuración."
)

# -----
# Validaciones del target
# -----
expected_mapping = (
    stage_07_experimental_config[
        "target"
    ][
        "expected_class_mapping"
    ]
)

assert len(expected_mapping) == 5, (
    "El target OPC debe contener cinco clases."
)

assert (
    stage_07_experimental_config["target"]["n_classes"]
    == len(expected_mapping)
), (
    "n_classes no coincide con el mapping esperado."
)

# -----
# Validar que la configuración pueda convertirse a JSON
# -----
json.dumps(
    stage_07_experimental_config,
    ensure_ascii=False
)

print("=" * 80)
print("VALIDACIÓN DE LA CONFIGURACIÓN")
print("=" * 80)

print("\nVentanas: OK")
print("Folds: OK")
print("Modelos: OK")
print("Métricas: OK")
print("Estructura JSON: OK")

```

```
print(
    "\nAdvertencia:"
    "\n- La codificación exacta del target todavía debe verificarse."
    "\n- La lista exacta de features todavía debe cargarse desde metadata."
)

print("\nEstado: CONFIGURACIÓN BASE VÁLIDA")
```

```
=====
VALIDACIÓN DE LA CONFIGURACIÓN
=====
```

Ventanas: OK
 Folds: OK
 Modelos: OK
 Métricas: OK
 Estructura JSON: OK

Advertencia:

- La codificación exacta del target todavía debe verificarse.
- La lista exacta de features todavía debe cargarse desde metadata.

Estado: CONFIGURACIÓN BASE VÁLIDA

Celda 12.4 — Guardado de los archivos

```
In [4]: # 12.4. Guardado de archivos de configuración
# =====

CONFIG_DIR = STAGE_07_DIRS["config"]

EXPERIMENTAL_CONFIG_PATH = (
    CONFIG_DIR
    / "stage_07_experimental_config.json"
)

FOLDS_CONFIG_PATH = (
    CONFIG_DIR
    / "stage_07_folds.csv"
)

MODELS_CONFIG_PATH = (
    CONFIG_DIR
    / "stage_07_models.csv"
)

METRICS_CONFIG_PATH = (
    CONFIG_DIR
    / "stage_07_metrics.csv"
)

# -----
# Guardar JSON principal
# -----
with EXPERIMENTAL_CONFIG_PATH.open(
    "w",
    encoding="utf-8"
```

```
) as file:

    json.dump(
        stage_07_experimental_config,
        file,
        indent=2,
        ensure_ascii=False,
    )

# -----
# Guardar tablas CSV
# -----

stage_07_folds.to_csv(
    FOLDS_CONFIG_PATH,
    index=False,
    encoding="utf-8",
)

stage_07_models.to_csv(
    MODELS_CONFIG_PATH,
    index=False,
    encoding="utf-8",
)

stage_07_metrics.to_csv(
    METRICS_CONFIG_PATH,
    index=False,
    encoding="utf-8",
)

print("=" * 80)
print("ARCHIVOS DE CONFIGURACIÓN GUARDADOS")
print("=" * 80)

print(f"\n1. Configuración principal:")
print(EXPERIMENTAL_CONFIG_PATH)

print(f"\n2. Folds:")
print(FOLDS_CONFIG_PATH)

print(f"\n3. Modelos:")
print(MODELS_CONFIG_PATH)

print(f"\n4. Métricas:")
print(METRICS_CONFIG_PATH)

print("\nEstado: GUARDADO COMPLETADO")
```

```
=====
ARCHIVOS DE CONFIGURACIÓN GUARDADOS
=====
```

1. Configuración principal:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\config\stage_07_experimental_config.json

2. Folds:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\config\stage_07_folds.csv

3. Modelos:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\config\stage_07_models.csv

4. Métricas:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models\config\stage_07_metrics.csv

Estado: GUARDADO COMPLETADO

Celda 12.5 — Verificación final leyendo los archivos

No basta con ejecutar `to_csv()` o `json.dump()`. Esta celda confirma que los archivos pueden volver a abrirse correctamente.

```
In [5]: # 12.5. Verificación final de archivos guardados
# =====

# -----
# Verificar existencia
# -----

expected_files = [
    EXPERIMENTAL_CONFIG_PATH,
    FOLDS_CONFIG_PATH,
    MODELS_CONFIG_PATH,
    METRICS_CONFIG_PATH,
]

missing_files = [
    path
    for path in expected_files
    if not path.exists()
]

if missing_files:
    raise FileNotFoundError(
        "No se encontraron los siguientes archivos:\n"
        + "\n".join(str(path) for path in missing_files)
    )

# -----
# Volver a cargar los archivos
# -----

with EXPERIMENTAL_CONFIG_PATH.open(
    "r",
```

```

        encoding="utf-8"
    ) as file:

        loaded_experimental_config = json.load(file)

loaded_folds = pd.read_csv(FOLDS_CONFIG_PATH)
loaded_models = pd.read_csv(MODELS_CONFIG_PATH)
loaded_metrics = pd.read_csv(METRICS_CONFIG_PATH)

# -----
# Verificaciones mínimas
# -----
assert loaded_experimental_config["stage"] == "stage_07"

assert (
    loaded_experimental_config["target"]["name"]
    == TARGET_NAME
)

assert len(loaded_folds) == 3

assert loaded_models["enabled"].sum() == 7

assert loaded_metrics["primary_metric"].sum() == 1

# -----
# Resumen final
# -----
print("=" * 80)
print("RESUMEN FINAL DEL STAGE_07 EXPERIMENTAL DESIGN")
print("=" * 80)

print(f"\nCarpeta principal:")
print(STAGE_07_ROOT)

print(f"\nCantidad de folds:")
print(len(loaded_folds))

print(f"\nCantidad de modelos habilitados:")
print(int(loaded_models["enabled"].sum()))

print(f"\nCantidad de métricas:")
print(len(loaded_metrics))

print(f"\nMétrica principal:")
print(
    loaded_metrics.loc[
        loaded_metrics["primary_metric"],
        "metric_name"
    ].iloc[0]
)

print("\nArchivos verificados:")

for path in expected_files:
    print(f"- {path.name}")

print(

```



```
"\nPendiente antes de construir las secuencias:"
"\n1. Verificar el mapping real del target OPC."
"\n2. Cargar la lista exacta de features desde los metadatos."
)

print("\nEstado final: CONFIGURACIÓN DEL STAGE_07 CREADA Y VERIFICADA")
```

=====

RESUMEN FINAL DEL STAGE_07 EXPERIMENTAL DESIGN

=====

Carpeta principal:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\07_mnq_models

Cantidad de folds:

3

Cantidad de modelos habilitados:

7

Cantidad de métricas:

8

Métrica principal:

macro_f1

Archivos verificados:

- stage_07_experimental_config.json
- stage_07_folds.csv
- stage_07_models.csv
- stage_07_metrics.csv

Pendiente antes de construir las secuencias:

1. Verificar el mapping real del target OPC.
2. Cargar la lista exacta de features desde los metadatos.

Estado final: CONFIGURACIÓN DEL STAGE_07 CREADA Y VERIFICADA